COVOF 2D — Recueil de Tutoriels et Cas de Validation Analytique

SUBCOVOF 2D — Recueil de Tutoriels et Cas de Validation Analytique

Équipe de Développement & Recherche COVOF 2D Direction Scientifique & Numérique

12 novembre 2026

Table des matières

1 Tutoriel 1 : Opérateur Laplacien avec Conditions de Dirichlet Homogènes 5
1.1 1. Objectifs Pédagogiques et Modules Logiciels Activés 5
1.1.1 1.1 Compétences Acquises 5
1.1.2 1.2 Modules Logiciels Sollicités 5
1.2 2. Formulation Physique et Solution Analytique Exacte 5
1.2.1 Solution analytique exacte 6
1.3 3. Paramètres Physiques, Numériques et Discrétisation 6
1.4 4. Fichier de Configuration JSON Pas à Pas 7
1.4.1 5.1 Prototype Solveur Direct (`case/cpp/cas_1_grid64_direct.json`) 7
1.4.2 5.2 Variante Solveur de Krylov Préconditionné (`case/cpp/cas_1_grid64_cg_ssor.json`) 7
1.5 5. Procédure d'Exécution (Moteurs C++17 et Fortran 2018) 8
1.6 6. Analyse Quantitative, Script Python et Visualisation ParaView 8
1.6.1 6.1 Critères de Vérification et Tolérances 8
1.6.2 6.2 Résultats Attendus et Visualisation ParaView 8

2 Tutoriel 2 : Opérateur Laplacien avec Conditions de Neumann Non Homogènes et Jauge de Pression 9
2.1 1. Objectifs Pédagogiques et Modules Logiciels Activés 9
2.1.1 1.1 Compétences Acquises 9
2.1.2 1.2 Modules Logiciels Sollicités 9
2.2 2. Formulation Physique et Solution Analytique Exacte 9
2.2.1 Propriété théorique fondamentale (exactitude machine) 10
2.3 3. Paramètres Physiques, Numériques et Discrétisation 10
2.4 4. Fichier de Configuration JSON Pas à Pas 11
2.4.1 Prototype `case/cpp/cas_2_neumann_grid64_direct.json` 11
2.5 5. Procédure d'Exécution (Moteurs C++17 et Fortran 2018) 11
2.6 6. Analyse Quantitative, Script Python et Visualisation ParaView 12
2.6.1 6.1 Critères de Vérification et Tolérances 12
2.6.2 6.2 Résultats Attendus et Visualisation ParaView 12

3 Tutoriel 3 : Écoulement de Poiseuille Plan Laminaire Stationnaire 13
3.1 1. Objectifs Pédagogiques et Modules Logiciels Activés 13
3.1.1 1.1 Compétences Acquises 13
3.1.2 1.2 Modules Logiciels Sollicités 13
3.2 2. Formulation Physique et Solution Analytique Exacte 13
3.2.1 Solution analytique exacte 14
3.2.2 Extrapolation quadratique à 3 points dans les mailles fantômes 14
3.3 3. Paramètres Physiques, Numériques et Discrétisation 14
3.4 4. Fichier de Configuration JSON Pas à Pas 14
3.4.1 Prototype `case/cpp/cas_3_poiseuille_grid64_al.json` 15
3.5 5. Procédure d'Exécution (Moteurs C++17 et Fortran 2018) 16

3.6 6. Analyse Quantitative, Script Python et Visualisation ParaView 16
3.6.1 6.1 Critères de Vérification et Tolérances 16
3.6.2 6.2 Résultats Attendus et Visualisation ParaView 16

4 Tutoriel 4 : Écoulement de Poiseuille Plan Instationnaire (Démarrage Impulsionnel) 17
4.1 1. Objectifs Pédagogiques et Modules Logiciels Activés 17
4.1.1 1.1 Compétences Acquises 17
4.1.2 1.2 Modules Logiciels Sollicités 17
4.2 2. Formulation Physique et Solution Analytique Exacte 17
4.2.1 Solution analytique exacte 18
4.3 3. Paramètres Physiques, Numériques et Discrétisation 18
4.4 4. Fichier de Configuration JSON Pas à Pas 18
4.4.1 Prototype `case/cpp/cas_4_unsteady_al_bicgstab2_ny64_dt0_01.json` . . . 19
4.5 5. Procédure d'Exécution (Moteurs C++17 et Fortran 2018) 19
4.6 6. Analyse Quantitative, Script Python et Visualisation ParaView 20
4.6.1 6.1 Critères de Vérification et Tolérances 20
4.6.2 6.2 Résultats Attendus et Visualisation ParaView 20

5 Tutoriel 5 : Écoulement de Poiseuille Plan à Viscosité Variable Exponentielle 21
5.1 1. Objectifs Pédagogiques et Modules Logiciels Activés 21
5.1.1 1.1 Compétences Acquises 21
5.1.2 1.2 Modules Logiciels Sollicités 21
5.2 2. Formulation Physique et Solution Analytique Exacte 21
5.2.1 Solution analytique exacte 22
5.3 3. Paramètres Physiques, Numériques et Discrétisation 22
5.4 4. Fichier de Configuration JSON Pas à Pas 22
5.4.1 Prototype `case/cpp/cas_5_var_visc_al_direct_ny128.json` 23
5.5 5. Procédure d'Exécution (Moteurs C++17 et Fortran 2018) 24
5.6 6. Analyse Quantitative, Script Python et Visualisation ParaView 24
5.6.1 6.1 Critères de Vérification et Tolérances 24
5.6.2 6.2 Résultats Attendus et Visualisation ParaView 24

6 Tutoriel 6 : Goutte Statique en Équilibre Capillaire et Loi de Young-Laplace 25
6.1 1. Objectifs Pédagogiques et Modules Logiciels Activés 25
6.1.1 1.1 Compétences Acquises 25
6.1.2 1.2 Modules Logiciels Sollicités 25
6.2 2. Formulation Physique et Solution Analytique Exacte 25
6.2.1 Loi analytique de Young-Laplace et régularisation CSF 26
6.3 3. Paramètres Physiques, Numériques et Discrétisation 26
6.4 4. Fichier de Configuration JSON Pas à Pas 26
6.4.1 Prototype `case/cpp/cas_6_axeA_wy_plic_n128.json` 26
6.5 5. Procédure d'Exécution (Moteurs C++17 et Fortran 2018) 27
6.6 6. Analyse Quantitative, Script Python et Visualisation ParaView 28
6.6.1 6.1 Critères de Vérification et Tolérances 28
6.6.2 6.2 Résultats Attendus et Visualisation ParaView 28

7 Tutoriel 7 : Advection d'une Tâche Circulaire dans un Tourbillon Cisaillé 29
7.1 1. Objectifs Pédagogiques et Modules Logiciels Activés 29
7.1.1 1.1 Compétences Acquises 29
7.1.2 1.2 Modules Logiciels Sollicités 29
7.2 2. Formulation Physique et Solution Analytique Exacte 29
7.3 3. Paramètres Physiques, Numériques et Discrétisation 30
7.4 4. Fichier de Configuration JSON Pas à Pas 30

7.4.1 Prototype `case/cpp/cas_7_weymouth_yue_plic_n128.json` 30
7.5 5. Procédure d'Exécution (Moteurs C++17 et Fortran 2018) 31
7.6 6. Analyse Quantitative, Script Python et Visualisation ParaView 32
7.6.1 6.1 Critères de Vérification et Tolérances 32
7.6.2 6.2 Résultats Attendus et Visualisation ParaView 32

8 Tutoriel 8 : Chute Libre d'un Cylindre Dense dans le Vide (Pénalisation de Brinkman) 33
8.1 1. Objectifs Pédagogiques et Modules Logiciels Activés 33
8.1.1 1.1 Compétences Acquises 33
8.1.2 1.2 Modules Logiciels Sollicités 33
8.2 2. Formulation Physique et Solution Analytique Exacte 33
8.2.1 Formulations numériques spécifiques mises en œuvre 34
8.3 3. Paramètres Physiques, Numériques et Discrétisation 34
8.4 4. Fichier de Configuration JSON Pas à Pas 35
8.4.1 Prototype `case/cpp/cas_8_fc_100x200.json` 35
8.5 5. Procédure d'Exécution (Moteurs C++17 et Fortran 2018) 36
8.6 6. Analyse Quantitative, Script Python et Visualisation ParaView 36
8.6.1 6.1 Critères de Vérification et Tolérances 36
8.6.2 6.2 Résultats Attendus et Visualisation ParaView 37

9 Tutoriel 9 : Sédimentation Confinée 2D d'une Inclusion Cylindrique et Loi de Faxén 38
9.1 1. Objectifs Pédagogiques et Modules Logiciels Activés 38
9.1.1 1.1 Compétences Acquises 38
9.1.2 1.2 Modules Logiciels Sollicités 38
9.2 2. Formulation Physique et Solution Analytique Exacte 38
9.2.0.1 Loi analytique de vitesse terminale de Faxén 39
9.3 3. Paramètres Physiques, Numériques et Discrétisation 39
9.4 4. Fichier de Configuration JSON Pas à Pas 39
9.4.1 Prototype `case/cpp/cas_9_sedimentation_faxen_n128.json` 39
9.5 5. Procédure d'Exécution (Moteurs C++17 et Fortran 2018) 40
9.6 6. Analyse Quantitative, Script Python et Visualisation ParaView 41
9.6.1 6.1 Critères de Vérification et Tolérances 41
9.6.2 6.2 Résultats Attendus et Visualisation ParaView 41

10 Tutoriel 10 : Compression d'Air par Injection d'Eau (Piston Compressible Bas Mach) 42
10.1 1. Objectifs Pédagogiques et Modules Logiciels Activés 42
10.1.1 1.1 Compétences Acquises 42
10.1.2 1.2 Modules Logiciels Sollicités 42
10.2 2. Formulation Physique et Solution Analytique Exacte 42
10.2.0.1 Configuration géométrique et loi analytique exacte 43
10.3 3. Paramètres Physiques, Numériques et Discrétisation 43
10.4 4. Fichier de Configuration JSON Pas à Pas 43
10.4.1 Prototype `case/cpp/cas_10_piston_compressible_n128.json` 44
10.5 5. Procédure d'Exécution (Moteurs C++17 et Fortran 2018) 45
10.6 6. Analyse Quantitative, Script Python et Visualisation ParaView 45
10.6.1 6.1 Critères de Vérification et Tolérances 45
10.6.2 6.2 Résultats Attendus et Visualisation ParaView 45

11 Tutoriel 11 : Compression Isotrope d'une Bulle d'Air en Cavité Carrée 46
11.1 1. Objectifs Pédagogiques et Modules Logiciels Activés 46
11.1.1 1.1 Compétences Acquises 46
11.1.2 1.2 Modules Logiciels Sollicités 46

11.2 2. Formulation Physique et Solution Analytique Exacte 46
11.2.0.1 Loi analytique d'évolution du rayon, de la masse volumique et de la pression 47
11.3 3. Paramètres Physiques, Numériques et Discrétisation 47
11.4 4. Fichier de Configuration JSON Pas à Pas 47
11.4.1 Prototype `case/cpp/cas_11_compression_bulle_n128.json` 47
11.5 5. Procédure d'Exécution (Moteurs C++17 et Fortran 2018) 48
11.6 6. Analyse Quantitative, Script Python et Visualisation ParaView 49
11.6.1 6.1 Critères de Vérification et Tolérances 49
11.6.2 6.2 Résultats Attendus et Visualisation ParaView 49

12 Tutoriel 12 : Rupture de Barrage 2D sur Fond Sec (Solution de Saint-Venant) 50
12.1 1. Objectifs Pédagogiques et Modules Logiciels Activés 50
12.1.1 1.1 Compétences Acquises 50
12.1.2 1.2 Modules Logiciels Sollicités 50
12.2 2. Formulation Physique et Solution Analytique Exacte 50
12.2.0.1 Solution analytique des caractéristiques de Saint-Venant 51
12.3 3. Paramètres Physiques, Numériques et Discrétisation 51
12.4 4. Fichier de Configuration JSON Pas à Pas 51
12.4.1 Prototype `case/cpp/cas_12_rupture_barrage_240x40.json` 51
12.5 5. Procédure d'Exécution (Moteurs C++17 et Fortran 2018) 52
12.6 6. Analyse Quantitative, Script Python et Visualisation ParaView 53
12.6.1 6.1 Critères de Vérification et Tolérances 53
12.6.2 6.2 Résultats Attendus et Visualisation ParaView 53

13 Tutoriel 13 : Oscillations Capillaires Amorties d'une Bulle de Gaz (Théorie de Lamb) 54
13.1 1. Objectifs Pédagogiques et Modules Logiciels Activés 54
13.1.1 1.1 Compétences Acquises 54
13.1.2 1.2 Modules Logiciels Sollicités 54
13.2 2. Formulation Physique et Solution Analytique Exacte 54
13.2.0.1 Relation de dispersion capillaire linéaire 55
13.3 3. Paramètres Physiques, Numériques et Discrétisation 55
13.4 4. Fichier de Configuration JSON Pas à Pas 55
13.4.1 Prototype `case/cpp/cas_13_oscillation_bulle_n128.json` 55
13.5 5. Procédure d'Exécution (Moteurs C++17 et Fortran 2018) 57
13.6 6. Analyse Quantitative, Script Python et Visualisation ParaView 57
13.6.1 6.1 Critères de Vérification et Tolérances 57
13.6.2 6.2 Résultats Attendus et Visualisation ParaView 57

14 Tutoriel 14 : Instabilité Interfaciale Diphasique de Rayleigh-Taylor (Dispersion Linéaire) 58
14.1 1. Objectifs Pédagogiques et Modules Logiciels Activés 58
14.1.1 1.1 Compétences Acquises 58
14.1.2 1.2 Modules Logiciels Sollicités 58
14.2 2. Formulation Physique et Solution Analytique Exacte 58
14.2.0.1 Relation de dispersion hydro-capillaire linéaire 59
14.3 3. Paramètres Physiques, Numériques et Discrétisation 59
14.4 4. Fichier de Configuration JSON Pas à Pas 59
14.4.1 Prototype `case/cpp/cas_14_rayleigh_taylor_sigma0_128x512.json` 59
14.5 5. Procédure d'Exécution (Moteurs C++17 et Fortran 2018) 61
14.6 6. Analyse Quantitative, Script Python et Visualisation ParaView 61
14.6.1 6.1 Critères de Vérification et Tolérances 61
14.6.2 6.2 Résultats Attendus et Visualisation ParaView 61

Chapitre 1

Tutoriel 1 : Opérateur Laplacien avec Conditions de Dirichlet Homogènes

Ce premier tutoriel constitue le point de départ fondamental de la vérification de COVOF 2D. Il permet de valider la discrétisation spatiale à l'ordre 2 de l'opérateur Laplacien scalaire 2D (équation de Poisson) avec conditions aux limites de Dirichlet homogènes sur maillage cartésien décalé MAC.

1.1 1. Objectifs Pédagogiques et Modules Logiciels Activés

1.1.1 1.1 Compétences Acquises

- Comprendre les fondements physiques et les équations aux dérivées partielles gouvernant l'écoulement.
- Configurer les paramètres numériques et conditions aux limites sur grille cartésienne décalée MAC.
- Exécuter le cas test avec les exécutables C++17 et Fortran 2018 (séquentiel et OpenMP).
- Quantifier avec rigueur la précision par rapport à la solution analytique exacte (L_2, L_∞).
- Produire et exploiter les visualisations scientifiques volumiques sous ParaView.

1.1.2 1.2 Modules Logiciels Sollicités

- **Moteur C++17 :** `Mesh2D`, `FlowFields`, `Config`, `PoissonSolver`, `BiCGSTAB2Solver`, `SSORPreconditioner`, `VTKWriter`.
- **Moteur Fortran 2018 :** `module_mesh`, `module_flow_fields`, `module_config`, `module_poisson`, `module_linear_solvers`, `module_preconditioners`, `module_vtk_writer`.

1.2 2. Formulation Physique et Solution Analytique Exacte

Équations Physiques et Configuration

Ce premier cas vérifie la consistance, la précision et l'ordre de convergence spatial de la discrétisation aux volumes finis de l'opérateur Laplacien bidimensionnel stationnaire (équation de Poisson) avec conditions aux limites de Dirichlet homogènes.

Sur le domaine rectangulaire $\Omega = [0, a] \times [0, b]$ de dimensions $a = L_x = 1.0\,\mathrm{m}$ et $b = L_y = 0.5\,\mathrm{m}$, on résout :

$$\Delta\Psi = \frac{\partial^2\Psi}{\partial x^2} + \frac{\partial^2\Psi}{\partial y^2} = S \quad \text{dans } \Omega =]0, a[\times]0, b[, \qquad \text{avec } S = 1.0\,\mathrm{m}^{-2},$$

soumis à la condition de Dirichlet homogène sur l'ensemble du bord $\partial\Omega$:

$$\Psi(0,y) = \Psi(a,y) = \Psi(x,0) = \Psi(x,b) = 0, \qquad \forall (x,y) \in \partial\Omega.$$

1.2.1 Solution analytique exacte

Par décomposition sur la base propre orthogonale de l'opérateur Laplacien avec conditions de Dirichlet sur $[0,a] \times [0,b]$, seuls les modes impairs $(2i+1, 2j+1)$ contribuent au développement en série de Fourier double :

$$\Psi_{\text{ex}}(x,y) = -\frac{16S}{\pi^4} \sum_{i=0}^{\infty} \sum_{j=0}^{\infty} \frac{\sin\left(\frac{(2i+1)\pi x}{a}\right) \sin\left(\frac{(2j+1)\pi y}{b}\right)}{(2i+1)(2j+1)\left[\left(\frac{2i+1}{a}\right)^2 + \left(\frac{2j+1}{b}\right)^2\right]}.$$

Dans les codes C++17 et Fortran 2018, cette série est évaluée aux centres des mailles $(x_I, y_J) = ((I-0.5)\Delta x, (J-0.5)\Delta y)$ avec 60×60 modes impairs $(i, j \in \{0, \dots, 59\})$. La valeur minimale au centre géométrique $(x,y) = (0.5, 0.25)\,\text{m}$ vaut $\Psi_{\min} \approx -0.0366$.

Sur un maillage cartésien uniforme à mailles carrées $\Delta x = a/N_x = \Delta y = b/N_y$, le Laplacien est approché par le schéma centré à 5 points d'ordre 2 :

$$\frac{\Psi_{I+1,J} - 2\Psi_{I,J} + \Psi_{I-1,J}}{\Delta x^2} + \frac{\Psi_{I,J+1} - 2\Psi_{I,J} + \Psi_{I,J-1}}{\Delta y^2} = S,$$

et la condition $\Psi = 0$ sur les parois situées à $\Delta x/2$ du premier centre de maille intérieur est imposée par antisymétrie linéaire dans la maille fantôme ($\Psi_{0,J} = -\Psi_{1,J}$), modifiant le stencil de bord en $(\Psi_{2,J} - 3\Psi_{1,J})/\Delta x^2$.

1.3 3. Paramètres Physiques, Numériques et Discrétisation

Physiques, Numériques et Matrice des Jeux de Paramètres

Catégorie	Paramètre	Valeur / Spécification
Géométrie	Domaine $[0, L_x] \times [0, L_y]$	$L_x = 1.0\,\text{m}$, $L_y = 0.5\,\text{m}$ (rapport d'aspect 2 : 1)
Séquence de grilles	$(N_x \times N_y)$ à mailles carrées	16×8, 32×16, 64×32, 128×64, 256×128
Pas d'espace	$\Delta x = \Delta y$	6.25×10^{-2}, 3.125×10^{-2}, 1.5625×10^{-2}, 7.8125×10^{-3}, $3.90625 \times 10^{-3}\,\text{m}$
Terme source	S (`source_term`)	$1.0\,\text{m}^{-2}$
Conditions aux limites	Gauche, Droite, Bas, Haut	`DIRICHLET` homogène ($\Psi = 0$)
Solveurs linéaires à tester	`solver.linear_solver`	`DIRECT` (Cholesky bande SPD), `CG`, `BICGSTAB2`, `BICGSTAB`, `GMRES`, `MINRES` (tol $= 10^{-12}$, max_iter $= 5000$)
Préconditionneurs à tester	`solver.precond_type`	`NONE`, `JACOBI`, `SSOR`, `ILUO`, `BLOCK_TRIANGULAR`

1.4 4. Fichier de Configuration JSON Pas à Pas

de Fichiers `.json` (C++17 et Fortran 2018)

1.4.1 5.1 Prototype Solveur Direct (`case/cpp/cas_1_grid64_direct.json`)

```
{
  "case_name": "cas_1_grid64_direct",
  "mode": "LAPLACIAN",
  "test_case": "CAS_1_DIRICHLET",
  "source_term": 1.0,
  "mesh": {
    "nx": 64,
    "ny": 32,
    "lx": 1.0,
    "ly": 0.5
  },
  "boundary_conditions": {
    "left":   { "type": "DIRICHLET", "u": 0.0, "v": 0.0 },
    "right":  { "type": "DIRICHLET", "u": 0.0, "v": 0.0 },
    "bottom": { "type": "DIRICHLET", "u": 0.0, "v": 0.0 },
    "top":    { "type": "DIRICHLET", "u": 0.0, "v": 0.0 }
  },
  "solver": {
    "type": "DIRECT",
    "linear_solver": "DIRECT",
    "precond_type": "NONE",
    "use_precond": false,
    "tol": 1e-12,
    "max_iter": 5000
  },
  "output": {
    "output_dir": "VALIDATION_TOTALE/validation_cas_1_laplacian/output/cpp/direct/grid_64x32",
    "case_name": "cas_1_grid64_direct",
    "vtk_interval": 1,
    "log_interval": 1,
    "write_csv_history": true
  }
}
```

1.4.2 5.2 Variante Solveur de Krylov Préconditionné (`case/cpp/cas_1_grid64_cg_ssor.json`)

Pour les variantes itératives et l'étude des préconditionneurs, remplacer le bloc `"solver"` par :

```
  "solver": {
    "type": "MONOLITHIC",
    "linear_solver": "CG",
    "precond_type": "SSOR",
    "use_precond": true,
    "tol": 1e-12,
    "max_iter": 5000
  }
```

(Pour Fortran 2018, générer les fichiers analogues dans `case/fortran/` avec `"output_dir": "VALIDATION_TOTALE/validation_cas_1_laplacian/output/fortran/..."`).

1.5 5. Procédure d'Exécution (Moteurs C++17 et Fortran 2018)

L'exécution s'effectue indifféremment à l'aide du binaire C++17 (`covof2d_cpp`) ou du binaire Fortran 2018 (`covof2d_fortran`), garantissant une stricte équivalence des résultats numériques.

```
# 1. Se placer dans le répertoire du cas
cd VALIDATION_TOTALE/Run_Valide-CASES/validation_cas_1_laplacian

# 2. Exécution avec le moteur C++17 (Séquentiel)
../../../COVOF_2D-C++/bin/covof2d_cpp case/cpp/cas_1_config.json

# 3. Exécution avec le moteur C++17 (Parallèle OpenMP)
export OMP_NUM_THREADS=4
../../../COVOF_2D-C++/bin/covof2d_cpp case/cpp/cas_1_config.json

# 4. Exécution avec le moteur Fortran 2018 (Séquentiel)
../../../COVOF_2D-FORTRAN/bin/covof2d_fortran case/fortran/cas_1_config.json

# 5. Exécution avec le moteur Fortran 2018 (Parallèle OpenMP)
export OMP_NUM_THREADS=4
../../../COVOF_2D-FORTRAN/bin/covof2d_fortran case/fortran/cas_1_config.json
```

1.6 6. Analyse Quantitative, Script Python et Visualisation ParaView

1.6.1 6.1 Critères de Vérification et Tolérances

à Mener (Précision, Intégrité et Performance)

1. **Précision et ordre de convergence spatial :**
   - Extraire les erreurs discrètes E_{L_1}, E_{L_2} et E_{L_∞} depuis le fichier `<output_dir>/<case_name>_summary.csv` ou en comparant le champ `psi` du fichier `.vtk` à la série de Fourier $\Psi_{\text{ex}}(x_I, y_J)$.
   - Vérifier que E_{L_1} décroît de 9.88×10^{-5} (16×8) à 3.69×10^{-7} (256×128) et que l'ordre de convergence de Richardson vérifie :
$$p_\alpha = \frac{\ln(E_{L_\alpha}(h_1)/E_{L_\alpha}(h_2))}{\ln(h_1/h_2)} \in [1.99, 2.03], \qquad \alpha \in \{1, 2, \infty\}.$$
2. **Intégrité et équivalence des solveurs linéaires et préconditionneurs :**
   - Vérifier que l'opérateur discret $-(\Delta_h \Psi)$ étant symétrique défini positif (SPD), les solveurs `DIRECT`, `CG`, `MINRES`, `BICGSTAB`, `BICGSTAB2` et `GMRES` convergent tous sans stagnation et fournissent exactement les mêmes normes d'erreur : $|E_{L_2}^{\text{Krylov}} - E_{L_2}^{\text{DIRECT}}| < 10^{-12}$.
   - Vérifier l'invariance de la solution finale vis-à-vis du préconditionneur (`NONE`, `JACOBI`, `SSOR`, `ILU0`, `BLOCK_TRIANGULAR`) : $\|\Psi_{P_1} - \Psi_{P_2}\|_\infty < 10^{-11}$.
   - Vérifier la symétrie géométrique quatre-quadrants de la solution : $|\Psi(x, y) - \Psi(a - x, y)| < 10^{-14}$ et $|\Psi(x, y) - \Psi(x, b - y)| < 10^{-14}$.
3. **Performance algorithmique et parité C++17 / Fortran 2018 :**
   - Dresser la table du nombre d'itérations N_{iter} et du temps CPU en fonction du conditionnement $\kappa(A) = \mathcal{O}(N_x^2)$ pour chaque couple (`linear_solver, precond_type`).
   - Vérifier que le solveur `DIRECT` (Cholesky bande de demi-largeur N_x) affiche un nombre d'itérations égal à 1.
   - Vérifier la parité stricte entre les binaires C++17 et Fortran 2018 : $\|\Psi^{\text{C++}} - \Psi^{\text{Fortran}}\|_\infty < 10^{-11}$.

1.6.2 6.2 Résultats Attendus et Visualisation ParaView

Chapitre 2

Tutoriel 2 : Opérateur Laplacien avec Conditions de Neumann Non Homogènes et Jauge de Pression

Ce tutoriel aborde la résolution d'une équation de Poisson sous conditions aux limites de Neumann pur. Il met en lumière deux aspects cruciaux du calcul CFD : le traitement rigoureux des flux pariétaux sur les faces décalées MAC et la levée du mode singulier à constante (jauge de pression) pour garantir l'unicité de la solution et la convergence du solveur.

2.1 1. Objectifs Pédagogiques et Modules Logiciels Activés

2.1.1 1.1 Compétences Acquises

- Comprendre les fondements physiques et les équations aux dérivées partielles gouvernant l'écoulement.
- Configurer les paramètres numériques et conditions aux limites sur grille cartésienne décalée MAC.
- Exécuter le cas test avec les exécutables C++17 et Fortran 2018 (séquentiel et OpenMP).
- Quantifier avec rigueur la précision par rapport à la solution analytique exacte (L_2, L_∞).
- Produire et exploiter les visualisations scientifiques volumiques sous ParaView.

2.1.2 1.2 Modules Logiciels Sollicités

- **Moteur C++17 :** `Mesh2D`, `FlowFields`, `Config`, `PoissonSolver`, `PressureGauge`, `PCGSolver`, `ILU0Preconditioner`.
- **Moteur Fortran 2018 :** `module_mesh`, `module_flow_fields`, `module_config`, `module_poisson`, `module_pressure_gauge`, `module_linear_solvers`, `module_preconditioners`.

2.2 2. Formulation Physique et Solution Analytique Exacte

Équations Physiques et Configuration

Ce cas vérifie le traitement des conditions aux limites de Neumann non homogènes (flux normaux imposés sur $\partial\Omega$) et la régularisation de jauge d'une matrice Laplacienne singulière dont le noyau est engendré par les fonctions constantes.

Sur le domaine carré unitaire $\Omega = [0,1] \times [0,1]\,\mathrm{m}^2$, on résout l'équation de Laplace homogène ($S = 0$) :

$$\Delta\Psi = \frac{\partial^2\Psi}{\partial x^2} + \frac{\partial^2\Psi}{\partial y^2} = 0 \quad \text{dans } \Omega,$$

avec les conditions aux limites de Neumann suivantes sur les quatre frontières :

$$\left.\frac{\partial\Psi}{\partial n}\right|_{x=0} = -\left.\frac{\partial\Psi}{\partial x}\right|_{x=0} = 0, \qquad \left.\frac{\partial\Psi}{\partial n}\right|_{x=1} = +\left.\frac{\partial\Psi}{\partial x}\right|_{x=1} = +2,$$

$$\left.\frac{\partial\Psi}{\partial n}\right|_{y=0} = -\left.\frac{\partial\Psi}{\partial y}\right|_{y=0} = 0, \qquad \left.\frac{\partial\Psi}{\partial n}\right|_{y=1} = +\left.\frac{\partial\Psi}{\partial y}\right|_{y=1} = -2.$$

La condition de compatibilité $\oint_{\partial\Omega} \frac{\partial\Psi}{\partial n}\,\mathrm{d}\gamma = 0 + 2(1) + 0 - 2(1) = 0$ est exactement vérifiée. En fixant la constante de jauge par $\Psi(0,0) = 0$, la solution analytique exacte est le polynôme harmonique quadratique :

$$\Psi_{\text{ex}}(x,y) = x^2 - y^2.$$

2.2.1 Propriété théorique fondamentale (exactitude machine)

Le développement de Taylor du schéma aux volumes finis centré à 5 points montre que l'erreur de troncature locale s'écrit :

$$\tau_{I,J} = \frac{\Delta x^2}{12}\frac{\partial^4\Psi_{\text{ex}}}{\partial x^4} + \frac{\Delta y^2}{12}\frac{\partial^4\Psi_{\text{ex}}}{\partial y^4} + \mathcal{O}(\Delta x^4) \equiv 0$$

car toutes les dérivées d'ordre ≥ 3 de $x^2 - y^2$ sont identiquement nulles. Après recalage de la moyenne de jauge :

$$\widetilde{\Psi}_{I,J} = \Psi_{I,J} - \frac{1}{N_x N_y}\sum_{p,q}\left(\Psi_{p,q} - \Psi_{\text{ex}}(x_p, y_q)\right),$$

l'erreur discrète se situe au niveau du zéro machine ($\sim 10^{-15} - 10^{-12}$) quelle que soit la finesse du maillage.

2.3 3. Paramètres Physiques, Numériques et Discrétisation

Physiques, Numériques et Matrice des Jeux de Paramètres

Catégorie	Paramètre	Valeur / Spécification
Géométrie	Domaine $[0, L_x] \times [0, L_y]$	$L_x = 1.0\,\mathrm{m}$, $L_y = 1.0\,\mathrm{m}$
Séquence de grilles	$(N_x \times N_y)$	16×16, 32×32, 64×64, 128×128, 256×256
Terme source	S (`source_term`)	$0.0\,\mathrm{m}^{-2}$
Flux de Neumann	`dpsi_dn` (Gauche, Droite, Bas, Haut)	0.0, +2.0, 0.0, −2.0
Solveurs linéaires à tester	`solver.linear_solver`	`DIRECT`, `CG`, `BICGSTAB2`, `MINRES`, `GMRES` (tol $= 10^{-14}$, max_iter $= 10000$)
Préconditionneurs à tester	`solver.precond_type`	`NONE`, `JACOBI`, `SSOR`, `ILU0`, `BLOCK_TRIANGULAR`

2.4 4. Fichier de Configuration JSON Pas à Pas

de Fichiers `.json` (C++17 et Fortran 2018)

2.4.1 Prototype `case/cpp/cas_2_neumann_grid64_direct.json`

```
{
  "case_name": "cas_2_neumann_grid64_direct",
  "mode": "LAPLACIAN",
  "test_case": "CAS_2_NEUMANN",
  "source_term": 0.0,
  "mesh": {
    "nx": 64,
    "ny": 64,
    "lx": 1.0,
    "ly": 1.0
  },
  "boundary_conditions": {
    "left":   { "type": "NEUMANN", "dpsi_dn":  0.0 },
    "right":  { "type": "NEUMANN", "dpsi_dn":  2.0 },
    "bottom": { "type": "NEUMANN", "dpsi_dn":  0.0 },
    "top":    { "type": "NEUMANN", "dpsi_dn": -2.0 }
  },
  "solver": {
    "type": "DIRECT",
    "linear_solver": "DIRECT",
    "precond_type": "NONE",
    "use_precond": false,
    "tol": 1e-14,
    "max_iter": 10000
  },
  "output": {
    "output_dir": "VALIDATION_TOTALE/validation_cas_2_neumann/output/cpp/direct/grid_64x64",
    "case_name": "cas_2_neumann_grid64_direct",
    "vtk_interval": 1,
    "log_interval": 1,
    "write_csv_history": true
  }
}
```

2.5 5. Procédure d'Exécution (Moteurs C++17 et Fortran 2018)

L'exécution s'effectue indifféremment à l'aide du binaire C++17 (`covof2d_cpp`) ou du binaire Fortran 2018 (`covof2d_fortran`), garantissant une stricte équivalence des résultats numériques.

```
# 1. Se placer dans le répertoire du cas
cd VALIDATION_TOTALE/Run_Valide-CASES/validation_cas_2_neumann

# 2. Exécution avec le moteur C++17 (Séquentiel)
../../../COVOF_2D-C++/bin/covof2d_cpp case/cpp/cas_2_config.json

# 3. Exécution avec le moteur C++17 (Parallèle OpenMP)
export OMP_NUM_THREADS=4
../../../COVOF_2D-C++/bin/covof2d_cpp case/cpp/cas_2_config.json

# 4. Exécution avec le moteur Fortran 2018 (Séquentiel)
../../../COVOF_2D-FORTRAN/bin/covof2d_fortran case/fortran/cas_2_config.json

# 5. Exécution avec le moteur Fortran 2018 (Parallèle OpenMP)
export OMP_NUM_THREADS=4
../../../COVOF_2D-FORTRAN/bin/covof2d_fortran case/fortran/cas_2_config.json
```

2.6 6. Analyse Quantitative, Script Python et Visualisation ParaView

2.6.1 6.1 Critères de Vérification et Tolérances

à Mener (Précision, Intégrité et Performance)

1. **Exactitude machine sur polynôme quadratique :**
   — Vérifier qu'avec le solveur `DIRECT`, les erreurs $E_{L_1}, E_{L_2}, E_{L_\infty}$ sont inférieures à 3×10^{-14} sur toutes les grilles de 16×16 à 256×256 (par ex. $E_{L_1}(16 \times 16) \approx 2.34 \times 10^{-16}$).
2. **Intégrité de la régularisation du noyau singulier (**$\ker(\Delta_N) = \mathrm{Vect}\{\mathbf{1}\}$**) :**
   — Vérifier que l'ancrage de jauge dans `DirectSolver` élimine le mode constant sans perturber les flux de Neumann aux bords ($\sum_{I,J}(\Delta_h \Psi)_{I,J} \Delta x \Delta y = 0$ à 10^{-16} près).
   — Vérifier que les solveurs de Krylov (`CG`, `MINRES`, `BICGSTAB2`, `GMRES`) couplés aux préconditionneurs `BLOCK_TRIANGULAR` (qui projette explicitement le résidu sur l'hyperplan à moyenne nulle $\sum_c r_p[c] = 0$), `JACOBI`, `SSOR` et `ILU0` atteignent une erreur $E_{L_2} \leq 10^{-12}$ sans dérive dans le noyau.
   — Vérifier l'antisymétrie diagonale $\Psi(y, x) = -\Psi(x, y)$ à 10^{-15} près.
3. **Performance et parité C++17 / Fortran 2018 :**
   — Comparer le nombre d'itérations et le temps CPU de chaque préconditionneur, et vérifier $\|\widetilde{\Psi}^{\mathrm{C++}} - \widetilde{\Psi}^{\mathrm{Fortran}}\|_\infty < 10^{-14}$.

2.6.2 6.2 Résultats Attendus et Visualisation ParaView

Chapitre 3

Tutoriel 3 : Écoulement de Poiseuille Plan Laminaire Stationnaire

Ce tutoriel traite de l'écoulement laminaire cisaillé 1D stationnaire en canal plan entre deux parois immobiles, entraîné par un gradient de pression longitudinal imposé. Il valide la discrétisation de Navier-Stokes incompressible et démontre l'intérêt capital de l'extrapolation quadratique à 3 points dans les mailles fantômes pour atteindre une précision machine sur la vitesse tangentielle.

3.1 1. Objectifs Pédagogiques et Modules Logiciels Activés

3.1.1 1.1 Compétences Acquises

- Comprendre les fondements physiques et les équations aux dérivées partielles gouvernant l'écoulement.
- Configurer les paramètres numériques et conditions aux limites sur grille cartésienne décalée MAC.
- Exécuter le cas test avec les exécutables C++17 et Fortran 2018 (séquentiel et OpenMP).
- Quantifier avec rigueur la précision par rapport à la solution analytique exacte (L_2, L_∞).
- Produire et exploiter les visualisations scientifiques volumiques sous ParaView.

3.1.2 1.2 Modules Logiciels Sollicités

- **Moteur C++17 :** `Mesh2D`, `FlowFields`, `Config`, `NavierStokesSolver`, `BiCGSTAB2Solver`, `SSORPreconditioner`, `VTKWriter`.
- **Moteur Fortran 2018 :** `module_mesh`, `module_flow_fields`, `module_config`, `module_navier_stokes`, `module_linear_solvers`, `module_preconditioners`, `module_vtk_writer`.

3.2 2. Formulation Physique et Solution Analytique Exacte

Équations Physiques et Configuration

Ce cas valide la résolution stationnaire des équations de Navier-Stokes incompressibles en canal plan laminaire et vérifie la propriété d'exactitude polynomiale d'ordre 2 du stencil visqueux couplé à l'extrapolation quadratique à 3 points dans les mailles fantômes de paroi.

Dans un canal plan $\Omega = [0, L] \times [0, H]$ de longueur $L = 1.0\,\mathrm{m}$ et de hauteur $H = 2e = 0.1\,\mathrm{m}$ ($e = 0.05\,\mathrm{m}$), un fluide newtonien de masse volumique $\rho = 1.0\,\mathrm{kg/m^3}$ et de viscosité $\mu = 1.0\times10^{-3}\,\mathrm{Pa}\cdot\mathrm{s}$ s'écoule avec une vitesse maximale d'entrée $u_{\max} = 1.5 \times 10^{-3}\,\mathrm{m/s}$ ($v = 0$, $\mathrm{Re}_H = \rho U_{\mathrm{moy}} H/\mu = 0.1$).

3.2.1 Solution analytique exacte

Le profil de vitesse longitudinale est purement parabolique en y et le champ de pression décroît linéairement en x :

$$u_{\text{ex}}(y) = \frac{4u_{\max}}{H^2} y(H-y) = 0.6\, y\,(0.1-y), \qquad v_{\text{ex}}(x,y) = 0,$$

$$-\frac{\mathrm{d}p_{\text{ex}}}{\mathrm{d}x} = \frac{8\mu u_{\max}}{H^2} = \frac{2\mu u_{\max}}{e^2} = 1.2\times 10^{-3}\,\text{Pa/m}, \qquad \Delta P_{\text{ex}} = 1.2\times 10^{-3}\,\text{Pa}.$$

3.2.2 Extrapolation quadratique à 3 points dans les mailles fantômes

Sur le maillage décalé MAC, les vitesses $u_{i,j}$ sont décalées de $\Delta y/2$ des parois $y = 0$ et $y = H$. COVOF 2D implémente l'extrapolation quadratique de Lagrange aux abscisses $(-\Delta y/2, 0, +\Delta y/2, +3\Delta y/2)$:

$$u_{i,0} = \frac{8}{3}u_{\text{wall}} - 2u_{i,1} + \frac{1}{3}u_{i,2}, \qquad u_{i,N_y+1} = \frac{8}{3}u_{\text{wall}} - 2u_{i,N_y} + \frac{1}{3}u_{i,N_y-1}.$$

En injectant cette relation dans le stencil visqueux centré au premier nœud intérieur $j = 1$, on obtient :

$$\frac{u_{i,0} - 2u_{i,1} + u_{i,2}}{\Delta y^2} = \frac{\frac{8}{3}u_{\text{wall}} - 4u_{i,1} + \frac{4}{3}u_{i,2}}{\Delta y^2},$$

qui est **exact** pour tout polynôme du second degré $u(y) \in \mathbb{P}_2$, garantissant une erreur spatiale nulle à la précision machine sur tout maillage.

3.3 3. Paramètres Physiques, Numériques et Discrétisation

Physiques, Numériques et Matrice des Jeux de Paramètres

Catégorie	Paramètre	Valeur / Spécification
Domaine	$[0, L_x] \times [0, L_y]$	$L_x = 1.0\,\text{m}$, $L_y = H = 0.1\,\text{m}$ ($e = 0.05\,\text{m}$)
Propriétés du fluide	$\rho, \mu, u_{\max}$	$\rho = 1.0\,\text{kg/m}^3$, $\mu = 10^{-3}\,\text{Pa}\cdot\text{s}$, $u_{\max} = 1.5\times 10^{-3}\,\text{m/s}$
Séquence de grilles	$(N_x \times N_y)$	16×16, 32×32, 64×64, 128×128
Conditions aux limites	Gauche, Droite, Bas/Haut	`INLET` (`"profile": "PARABOLIC"`), `OUTLET` ($p = 0$), `DIRICHLET` ($u = v = 0$)
Solveurs de couplage	`solver.type`	`DIRECT`, `AUGMENTED_LAGRANGIAN` ($r_{\text{al}} \in \{0, 1, 100\}$), `ADAPTATIVE_AUGMENTED_LAGRANGIAN` ($K_{\text{aal}} = 100$), `MONOLITHIC`, `STANDARD_PROJECTION`
Solveurs linéaires & Précond.	`linear_solver` / `precond_type`	`DIRECT`, `BICGSTAB2`, `BICGSTAB`, `GMRES`, `MINRES` avec `NONE`, `JACOBI`, `SSOR`, `ILU0`, `BLOCK_TRIANGULAR`

3.4 4. Fichier de Configuration JSON Pas à Pas

de Fichiers `.json` (C++17 et Fortran 2018)

3.4.1 Prototype `case/cpp/cas_3_poiseuille_grid64_al.json`

```
{
  "case_name": "cas_3_poiseuille_grid64_al",
  "mode": "NAVIER_STOKES",
  "test_case": "POISEUILLE",
  "u_max_poiseuille": 0.0015,
  "steady_state_tol": 1e-13,
  "mesh": {
    "nx": 64,
    "ny": 64,
    "lx": 1.0,
    "ly": 0.1
  },
  "physics": {
    "enable_two_phase": false,
    "enable_surface_tension": false,
    "enable_convection": true,
    "convective_scheme": "UPWIND",
    "gx": 0.0,
    "gy": 0.0,
    "fluid1": {
      "rho": 1.0,
      "mu": 0.001
    }
  },
  "time": {
    "dt": 0.1,
    "t_final": 0.5,
    "cfl_max": 0.5,
    "adaptive_dt": false
  },
  "boundary_conditions": {
    "left":   { "type": "INLET",     "u": 0.0015, "v": 0.0, "profile": "PARABOLIC" },
    "right":  { "type": "OUTLET",    "u": 0.0,    "v": 0.0 },
    "bottom": { "type": "DIRICHLET", "u": 0.0,    "v": 0.0 },
    "top":    { "type": "DIRICHLET", "u": 0.0,    "v": 0.0 }
  },
  "solver": {
    "type": "AUGMENTED_LAGRANGIAN",
    "linear_solver": "BICGSTAB2",
    "precond_type": "BLOCK_TRIANGULAR",
    "use_precond": true,
    "tol": 1e-12,
    "max_iter": 1000,
    "r_al": 1.0,
    "uzawa_iter": 1
  },
  "output": {
    "output_dir":
    ↪  "VALIDATION_TOTALE/validation_cas_3_poiseuille/output/cpp/cas_3_poiseuille_grid64_al",
    "case_name": "cas_3_poiseuille_grid64_al",
    "vtk_interval": 5,
    "log_interval": 1,
    "write_csv_history": true
  }
}
```

3.5 5. Procédure d'Exécution (Moteurs C++17 et Fortran 2018)

L'exécution s'effectue indifféremment à l'aide du binaire C++17 (`covof2d_cpp`) ou du binaire Fortran 2018 (`covof2d_fortran`), garantissant une stricte équivalence des résultats numériques.

```
# 1. Se placer dans le répertoire du cas
cd VALIDATION_TOTALE/Run_Valide-CASES/validation_cas_3_poiseuille

# 2. Exécution avec le moteur C++17 (Séquentiel)
../../../COVOF_2D-C++/bin/covof2d_cpp case/cpp/cas_3_config.json

# 3. Exécution avec le moteur C++17 (Parallèle OpenMP)
export OMP_NUM_THREADS=4
../../../COVOF_2D-C++/bin/covof2d_cpp case/cpp/cas_3_config.json

# 4. Exécution avec le moteur Fortran 2018 (Séquentiel)
../../../COVOF_2D-FORTRAN/bin/covof2d_fortran case/fortran/cas_3_config.json

# 5. Exécution avec le moteur Fortran 2018 (Parallèle OpenMP)
export OMP_NUM_THREADS=4
../../../COVOF_2D-FORTRAN/bin/covof2d_fortran case/fortran/cas_3_config.json
```

3.6 6. Analyse Quantitative, Script Python et Visualisation ParaView

3.6.1 6.1 Critères de Vérification et Tolérances

à Mener (Précision, Intégrité et Performance)

1. **Exactitude machine du profil de vitesse et du gradient de pression :**
   — Vérifier que sur toutes les grilles (16×16 à 128×128), l'erreur sur u satisfait $E_{L_1}, E_{L_2}, E_{L_\infty} < 10^{-12}\,\mathrm{m/s}$ (et $\sim 10^{-18}\,\mathrm{m/s}$ avec `DIRECT`), et que la vitesse transversale vérifie $\|v\|_\infty < 10^{-15}\,\mathrm{m/s}$.
   — Vérifier que le gradient axial de pression restitué par régression linéaire sur $p(x, H/2)$ vaut $-\mathrm{d}p/\mathrm{d}x = 1.200000 \times 10^{-3}\,\mathrm{Pa/m}$ avec une erreur relative $< 10^{-11}$.
2. **Intégrité des schémas convectifs, des couplages et des préconditionneurs :**
   — Vérifier l'invariance vis-à-vis du schéma convectif (`UPWIND`, `CENTERED`, `TVD_SUPERBEE`) et de la linéarisation (`PICARD`, `ADAMS_BASHFORTH`, `NEWTON`) puisque $\partial u_{\mathrm{ex}}/\partial x = 0$ et $v_{\mathrm{ex}} = 0$.
   — Comparer la vitesse de convergence entre `DIRECT`, `MONOLITHIC`, `AUGMENTED_LAGRANGIAN` ($r_{\mathrm{al}} \in \{0, 1, 100\}$), `ADAPTATIVE_AUGMENTED_LAGRANGIAN` et `STANDARD_PROJECTION`.
   — Vérifier que le préconditionneur `BLOCK_TRIANGULAR` surpasse `JACOBI`, `SSOR` et `ILU0` sur le système point-selle complet (u, v, p).
3. **Parité C++17 / Fortran 2018 :** Vérifier que $\|u^{\mathrm{C++}} - u^{\mathrm{Fortran}}\|_\infty < 10^{-14}\,\mathrm{m/s}$ et $\|p^{\mathrm{C++}} - p^{\mathrm{Fortran}}\|_\infty < 10^{-14}\,\mathrm{Pa}$.

3.6.2 6.2 Résultats Attendus et Visualisation ParaView

Chapitre 4

Tutoriel 4 : Écoulement de Poiseuille Plan Instationnaire (Démarrage Impulsionnel)

Ce tutoriel aborde le régime transitoire d'établissement visqueux d'un écoulement de Poiseuille plan à partir du repos sous l'effet soudain d'un gradient de pression. La solution analytique en série de Fourier permet de valider avec une haute précision les schémas d'intégration en temps (Euler implicite, Crank-Nicolson) et la stabilité temporelle.

4.1 1. Objectifs Pédagogiques et Modules Logiciels Activés

4.1.1 1.1 Compétences Acquises

- Comprendre les fondements physiques et les équations aux dérivées partielles gouvernant l'écoulement.
- Configurer les paramètres numériques et conditions aux limites sur grille cartésienne décalée MAC.
- Exécuter le cas test avec les exécutables C++17 et Fortran 2018 (séquentiel et OpenMP).
- Quantifier avec rigueur la précision par rapport à la solution analytique exacte (L_2, L_∞).
- Produire et exploiter les visualisations scientifiques volumiques sous ParaView.

4.1.2 1.2 Modules Logiciels Sollicités

- **Moteur C++17 :** `Mesh2D`, `FlowFields`, `TimeIntegrator`, `NavierStokesSolver`, `BiCGSTAB2Solver`, `SSORPreconditioner`.
- **Moteur Fortran 2018 :** `module_mesh`, `module_flow_fields`, `module_time`, `module_navier_stokes`, `module_linear_solvers`, `module_preconditioners`.

4.2 2. Formulation Physique et Solution Analytique Exacte

Équations Physiques et Configuration

Ce cas étudie l'établissement transitoire d'un écoulement de Poiseuille plan partant du repos sous l'action brusque d'un terme source de pression axial constant. Il valide le couplage entre le schéma d'intégration temporelle (`EULER` d'ordre 1 ou `GEAR`/BDF2 d'ordre 2) et la discrétisation spatiale visqueuse d'ordre 2.

Dans un canal périodique en x de dimensions $L_x = 0.16\,\mathrm{m}$ et $H = L_y = 2e = 0.16\,\mathrm{m}$ ($e = 0.08\,\mathrm{m}$), un fluide au repos initial $u(x, y, 0) = 0, v(x, y, 0) = 0$ de propriétés $\rho = 1.0\,\mathrm{kg/m^3}$ et $\mu = 0.01\,\mathrm{Pa \cdot s}$ ($\nu = 0.01\,\mathrm{m^2/s}$) est soumis à l'instant $t = 0^+$ à un forçage axial constant :

$$k = -\frac{\partial p}{\partial x} = \rho g_x = 0.0555\,\mathrm{N/m^3}.$$

L'équation d'évolution de la vitesse longitudinale $u(y, t)$ s'écrit :

$$\rho \frac{\partial u}{\partial t} = k + \mu \frac{\partial^2 u}{\partial y^2}, \qquad u(0, t) = u(H, t) = 0, \qquad u(y, 0) = 0.$$

4.2.1 Solution analytique exacte

Par séparation des variables, la solution exacte est la superposition du profil parabolique asymptotique ($u_{\max,\infty} = \frac{kH^2}{8\mu} = 0.01776\,\mathrm{m/s}$) et d'une série de Fourier amortie exponentiellement :

$$u_{\mathrm{ex}}(y, t) = \frac{k}{2\mu} y(H - y) - \frac{4kH^2}{\mu\pi^3} \sum_{m=0}^{\infty} \frac{1}{(2m+1)^3} \sin\left(\frac{(2m+1)\pi y}{H}\right) \exp\left(-\nu \left(\frac{(2m+1)\pi}{H}\right)^2 t\right).$$

Avec le schéma d'Euler implicite d'ordre 1 en temps et le schéma centré d'ordre 2 en espace, l'erreur globale obéit à la loi de compétition asymptotique :

$$\epsilon_{\mathrm{EULER}}(\Delta t, \Delta y) = \alpha_1 \Delta t + \beta_1 \Delta t \Delta y + \gamma \Delta y^2,$$

tandis qu'avec le schéma de Gear/BDF2 d'ordre 2 en temps, le terme temporel dominant devient quadratique :

$$\epsilon_{\mathrm{GEAR}}(\Delta t, \Delta y) = \alpha_2 \Delta t^2 + \gamma \Delta y^2.$$

4.3 3. Paramètres Physiques, Numériques et Discrétisation

Physiques, Numériques et Matrice des Jeux de Paramètres

Catégorie	Paramètre	Valeur / Spécification
Domaine	$[0, L_x] \times [0, L_y]$	$L_x = 0.16\,\mathrm{m}$, $L_y = H = 0.16\,\mathrm{m}$ ($e = 0.08\,\mathrm{m}$)
Propriétés et forçage	ρ, μ, g_x	$\rho = 1.0\,\mathrm{kg/m^3}$, $\mu = 0.01\,\mathrm{Pa \cdot s}$, $g_x = 0.0555\,\mathrm{m/s^2}$, $g_y = 0.0$
Conditions aux limites	Gauche/Droite ; Bas/Haut	`PERIODIC` en x ; `DIRICHLET` ($u = v = 0$) en $y = 0$ et $y = H$
Campagne 1 (Matrice $N_y \times \Delta t$)	$t_{\mathrm{final}} = 1.0\,\mathrm{s}$	$N_y \in \{16, 32, 64, 128, 256\} \times \Delta t \in \{10^{-1}, 10^{-2}, 10^{-3}, 10^{-4}\}\,\mathrm{s}$ ($N_x = 16$)
Campagne 2 (Profils transitoires)	$t_{\mathrm{final}} = 2.0\,\mathrm{s}$	$N_y = 64$, $\Delta t \in \{0.2, 0.1, 0.05, 0.01\}\,\mathrm{s}$, instants $t \in \{0.5, 1.0, 1.5, 2.0\}\,\mathrm{s}$
Schémas et Solveurs	Temps, Couplage, Krylov	`EULER` vs `GEAR` ; `AL+BICGSTAB2`, `AL+DIRECT`, `AAL+BICGSTAB2`, `MONO+BICGSTAB2`, `MONO+DIRECT`

4.4 4. Fichier de Configuration JSON Pas à Pas

de Fichiers `.json` (C++17 et Fortran 2018)

4.4.1 Prototype `case/cpp/cas_4_unsteady_al_bicgstab2_ny64_dt0_01.json`

```
{
  "mode": "NAVIER_STOKES",
  "test_case": "POISEUILLE_unsteady",
  "mesh": {
    "nx": 16,
    "ny": 64,
    "lx": 0.16,
    "ly": 0.16
  },
  "physics": {
    "enable_two_phase": false,
    "enable_surface_tension": false,
    "enable_convection": false,
    "gx": 0.0555,
    "gy": 0.0,
    "fluid1": {
      "rho": 1.0,
      "mu": 0.01
    }
  },
  "time": {
    "time_scheme": "EULER",
    "dt": 0.01,
    "t_final": 1.0,
    "cfl_max": 0.5,
    "adaptive_dt": false
  },
  "boundary_conditions": {
    "left":   { "type": "PERIODIC",  "u": 0.0, "v": 0.0 },
    "right":  { "type": "PERIODIC",  "u": 0.0, "v": 0.0 },
    "bottom": { "type": "DIRICHLET", "u": 0.0, "v": 0.0 },
    "top":    { "type": "DIRICHLET", "u": 0.0, "v": 0.0 }
  },
  "solver": {
    "type": "AUGMENTED_LAGRANGIAN",
    "linear_solver": "BICGSTAB2",
    "tol": 1e-12,
    "max_iter": 5000,
    "r_al": 0.0,
    "uzawa_iter": 1,
    "use_precond": true,
    "precond_type": "BLOCK_TRIANGULAR"
  },
  "output": {
    "output_dir":
    ↪  "VALIDATION_TOTALE/validation_cas_4_poiseuille_instat/output/cpp/al_bicgstab2/ny64_dt0_01",
    "case_name": "cas_4_unsteady_al_bicgstab2_ny64_dt0_01",
    "vtk_interval": 25,
    "log_interval": 10,
    "write_csv_history": true
  }
}
```

4.5 5. Procédure d'Exécution (Moteurs C++17 et Fortran 2018)

L'exécution s'effectue indifféremment à l'aide du binaire C++17 (`covof2d_cpp`) ou du binaire Fortran 2018 (`covof2d_fortran`), garantissant une stricte équivalence des résultats numériques.

```
# 1. Se placer dans le répertoire du cas
cd VALIDATION_TOTALE/Run_Valide-CASES/validation_cas_4_poiseuille_instat

# 2. Exécution avec le moteur C++17 (Séquentiel)
../../../COVOF_2D-C++/bin/covof2d_cpp case/cpp/cas_4_config.json

# 3. Exécution avec le moteur C++17 (Parallèle OpenMP)
export OMP_NUM_THREADS=4
../../../COVOF_2D-C++/bin/covof2d_cpp case/cpp/cas_4_config.json

# 4. Exécution avec le moteur Fortran 2018 (Séquentiel)
../../../COVOF_2D-FORTRAN/bin/covof2d_fortran case/fortran/cas_4_config.json

# 5. Exécution avec le moteur Fortran 2018 (Parallèle OpenMP)
export OMP_NUM_THREADS=4
../../../COVOF_2D-FORTRAN/bin/covof2d_fortran case/fortran/cas_4_config.json
```

4.6 6. Analyse Quantitative, Script Python et Visualisation ParaView

4.6.1 6.1 Critères de Vérification et Tolérances

à Mener (Précision, Intégrité et Performance)

1. **Matrice de convergence spatio-temporelle 5×4 à $t = 1.0\,\mathrm{s}$:**
   — Pour $\Delta t = 10^{-4}\,\mathrm{s}$, vérifier que l'erreur spatiale décroît en $\mathcal{O}(\Delta y^2)$ de 6.68×10^{-5} ($N_y = 16$) à 4.21×10^{-6} ($N_y = 64$) et 2.84×10^{-7} ($N_y = 256$).
   — Vérifier que l'ordre temporel apparent de `EULER` entre $\Delta t = 0.1\,\mathrm{s}$ et $\Delta t = 10^{-4}\,\mathrm{s}$ croît de 0.21 ($N_y = 16$, saturé par Δy^2) à 0.98 ($N_y = 256$, régime asymptotique d'ordre 1).
2. **Vérification comparative des schémas temporels `EULER` vs `GEAR` (BDF2) :**
   — Sur la grille fine $N_y = 256$, exécuter $\Delta t \in \{0.1, 0.05, 0.025, 0.01\}\,\mathrm{s}$ avec `"time_scheme": "GEAR"` et vérifier que l'erreur temporelle décroît avec un ordre proche de 2 et divise l'erreur d'Euler par un facteur > 10 dès $\Delta t = 0.01\,\mathrm{s}$.
3. **Performance du cache de factorisation de `DirectSolver` et des préconditionneurs :**
   — Vérifier que `DirectSolver` réutilise la factorisation LU mise en cache lorsque Δt et ρ, μ sont constants (`is_factored == true`), accélérant les 10 000 pas de temps à $\Delta t = 10^{-4}\,\mathrm{s}$.
   — Vérifier l'invariance des 4 combinaisons (`AL+BICGSTAB2`, `AL+DIRECT`, `MONO+BICGSTAB2`, `MONO+DIRECT`) et la parité C++/Fortran ($|E_{L_1}^{\mathrm{C++}} - E_{L_1}^{\mathrm{Fortran}}| < 10^{-15}$).

4.6.2 6.2 Résultats Attendus et Visualisation ParaView

Chapitre 5

Tutoriel 5 : Écoulement de Poiseuille Plan à Viscosité Variable Exponentielle

Ce tutoriel explore l'effet d'une hétérogénéité spatiale continue de la viscosité dynamique mu(y) = mu_0 exp(beta y) sur un écoulement de Poiseuille. Il valide la moyenne harmonique de la viscosité aux faces des volumes de contrôle et vérifie le profil asymétrique de vitesse ainsi que le déplacement du pic de vitesse vers la zone de moindre viscosité.

5.1 1. Objectifs Pédagogiques et Modules Logiciels Activés

5.1.1 1.1 Compétences Acquises

- Comprendre les fondements physiques et les équations aux dérivées partielles gouvernant l'écoulement.
- Configurer les paramètres numériques et conditions aux limites sur grille cartésienne décalée MAC.
- Exécuter le cas test avec les exécutables C++17 et Fortran 2018 (séquentiel et OpenMP).
- Quantifier avec rigueur la précision par rapport à la solution analytique exacte (L_2, L_∞).
- Produire et exploiter les visualisations scientifiques volumiques sous ParaView.

5.1.2 1.2 Modules Logiciels Sollicités

- **Moteur C++17 :** `Mesh2D`, `FlowFields`, `VariableViscosity`, `NavierStokesSolver`, `BiCGSTAB2Solver`, `SSORPreconditioner`.
- **Moteur Fortran 2018 :** `module_mesh`, `module_flow_fields`, `module_viscosity`, `module_navier_stokes`, `module_linear_solvers`, `module_preconditioners`.

5.2 2. Formulation Physique et Solution Analytique Exacte

Équations Physiques et Configuration

Ce cas vérifie la discrétisation conservative du terme de diffusion visqueuse $\nabla \cdot [\mu(\mathbf{x})(\nabla \mathbf{u} + \nabla \mathbf{u}^T)]$ lorsque la viscosité dynamique varie fortement dans l'espace.

Dans un canal plan $\Omega = [0, L_x] \times [0, L_y]$ avec $L_x = 0.05\,\text{m}$ et $L_y = 2e = 0.02\,\text{m}$ (demi-largeur $e = 0.01\,\text{m}$, coordonnée transversale centrée $z = y - e \in [-e, +e]$), la viscosité dynamique suit une loi exponentielle

symétrique :

$$\mu(z) = \mu_0 \exp\left(a\frac{z^2}{e^2}\right), \qquad \text{avec } \mu_0 = 1.85 \times 10^{-5}\,\text{Pa}\cdot\text{s}, \quad a = 2.0, \quad e = 0.01\,\text{m}.$$

Le rapport de viscosité entre les parois ($|z| = e$) et l'axe central ($z = 0$) atteint $\mu(\pm e)/\mu_0 = e^2 \approx 7.389$. Sous un forçage axial constant $G_x = -\partial p/\partial x = \rho g_x = 0.294207317\,\text{Pa/m}$, l'équation différentielle transversale s'écrit :

$$\frac{\mathrm{d}}{\mathrm{d}z}\left(\mu(z)\frac{\mathrm{d}u}{\mathrm{d}z}\right) = -G_x, \qquad u(-e) = u(+e) = 0.$$

5.2.1 Solution analytique exacte

Une première intégration avec $\frac{\mathrm{d}u}{\mathrm{d}z}(0) = 0$ donne la contrainte de cisaillement linéaire $\tau_{xz}(z) = \mu(z)\frac{\mathrm{d}u}{\mathrm{d}z} = -G_x z$. En remarquant que $\frac{\mathrm{d}}{\mathrm{d}z}\left[\exp(-az^2/e^2)\right] = -\frac{2az}{e^2}\exp(-az^2/e^2)$, une seconde intégration fournit le profil analytique gaussien tronqué :

$$u_{\text{ex}}(z) = \frac{G_x e^2}{2a\mu_0}\left[\exp\left(-a\frac{z^2}{e^2}\right) - \exp(-a)\right],$$

de vitesse maximale au centre $u_{\max} = u_{\text{ex}}(0) = \frac{G_x e^2}{2a\mu_0}(1 - e^{-2}) \approx 0.343771\,\text{m/s}$. Sur le maillage décalé MAC, les flux de cisaillement aux faces Nord et Sud des volumes de contrôle de $u_{i,j}$ sont évalués avec la viscosité aux nœuds de vorticité $y_j = j\Delta y$:

$$\left.\frac{\partial}{\partial y}\left(\mu\frac{\partial u}{\partial y}\right)\right|_{i,j} \approx \frac{1}{\Delta y}\left[\mu_{\text{N}}\frac{u_{i,j+1} - u_{i,j}}{\Delta y} - \mu_{\text{S}}\frac{u_{i,j} - u_{i,j-1}}{\Delta y}\right],$$

ce qui assure une précision d'ordre 2 ($\mathcal{O}(\Delta y^2)$).

5.3 3. Paramètres Physiques, Numériques et Discrétisation

Physiques, Numériques et Matrice des Jeux de Paramètres

Catégorie	Paramètre	Valeur / Spécification
Domaine	$[0, L_x] \times [0, L_y]$	$L_x = 0.05\,\text{m}$, $L_y = 0.02\,\text{m}$ ($e = 0.01\,\text{m}$)
Loi de viscosité	`viscosity_model: "EXPONENTIAL"`	$\mu_0 = 1.85 \times 10^{-5}\,\text{Pa}\cdot\text{s}$, $a = 2.0$, $e = 0.01\,\text{m}$, $\rho = 1.0\,\text{kg/m}^3$
Forçage axial	g_x, g_y	$g_x = 0.294207317\,\text{m/s}^2$, $g_y = 0.0$
Séquence de grilles	$N_x \times N_y$	$N_x = 16$, $N_y \in \{16, 32, 64, 128, 256, 512\}$
Intégration temporelle	Δt, t_{final}	$\Delta t = 0.5\,\text{s}$, $t_{\text{final}} = 50.0\,\text{s}$, steady_state_tol $= 10^{-12}$
Solveurs testés	Formulations	`AL`, `AAL`, `MONO` avec `DIRECT`, `BICGSTAB2`, `BLOCK_TRIANGULAR`, `SSOR`, `ILU0`

5.4 4. Fichier de Configuration JSON Pas à Pas

de Fichiers `.json` (C++17 et Fortran 2018)

5.4.1 Prototype `case/cpp/cas_5_var_visc_al_direct_ny128.json`

```
{
  "mode": "NAVIER_STOKES",
  "test_case": "CAS_5_VAR_MU",
  "steady_state_tol": 1e-12,
  "mesh": {
    "nx": 16,
    "ny": 128,
    "lx": 0.05,
    "ly": 0.02
  },
  "physics": {
    "enable_two_phase": false,
    "enable_surface_tension": false,
    "enable_convection": false,
    "viscosity_model": "EXPONENTIAL",
    "visc_a": 2.0,
    "visc_e": 0.01,
    "visc_mu0": 1.85e-05,
    "gx": 0.294207317,
    "gy": 0.0,
    "fluid1": {
      "rho": 1.0,
      "mu": 1.85e-05
    }
  },
  "time": {
    "dt": 0.5,
    "t_final": 50.0,
    "cfl_max": 0.5,
    "adaptive_dt": false
  },
  "boundary_conditions": {
    "left":   { "type": "PERIODIC",  "u": 0.0, "v": 0.0 },
    "right":  { "type": "PERIODIC",  "u": 0.0, "v": 0.0 },
    "bottom": { "type": "DIRICHLET", "u": 0.0, "v": 0.0 },
    "top":    { "type": "DIRICHLET", "u": 0.0, "v": 0.0 }
  },
  "solver": {
    "type": "AUGMENTED_LAGRANGIAN",
    "linear_solver": "DIRECT",
    "tol": 1e-12,
    "max_iter": 5000,
    "r_al": 0.0,
    "uzawa_iter": 1,
    "use_precond": false,
    "precond_type": "NONE",
    "steady_state_tol": 1e-12
  },
  "output": {
    "output_dir":
    ↪  "VALIDATION_TOTALE/validation_cas_5_poiseuille_visc_variable/output/cpp/al_direct/ny128",
    "case_name": "cas_5_var_visc_al_direct_ny128",
    "vtk_interval": 200,
    "log_interval": 50,
    "write_csv_history": true
  }
}
```

5.5 5. Procédure d'Exécution (Moteurs C++17 et Fortran 2018)

L'exécution s'effectue indifféremment à l'aide du binaire C++17 (`covof2d_cpp`) ou du binaire Fortran 2018 (`covof2d_fortran`), garantissant une stricte équivalence des résultats numériques.

```
# 1. Se placer dans le répertoire du cas
cd VALIDATION_TOTALE/Run_Valide-CASES/validation_cas_5_poiseuille_visc_variable

# 2. Exécution avec le moteur C++17 (Séquentiel)
../../../COVOF_2D-C++/bin/covof2d_cpp case/cpp/cas_5_config.json

# 3. Exécution avec le moteur C++17 (Parallèle OpenMP)
export OMP_NUM_THREADS=4
../../../COVOF_2D-C++/bin/covof2d_cpp case/cpp/cas_5_config.json

# 4. Exécution avec le moteur Fortran 2018 (Séquentiel)
../../../COVOF_2D-FORTRAN/bin/covof2d_fortran case/fortran/cas_5_config.json

# 5. Exécution avec le moteur Fortran 2018 (Parallèle OpenMP)
export OMP_NUM_THREADS=4
../../../COVOF_2D-FORTRAN/bin/covof2d_fortran case/fortran/cas_5_config.json
```

5.6 6. Analyse Quantitative, Script Python et Visualisation ParaView

5.6.1 6.1 Critères de Vérification et Tolérances

à Mener (Précision, Intégrité et Performance)

1. **Convergence spatiale d'ordre 2 et linéarité du flux de cisaillement :**
   — Vérifier que E_{L_1} décroît de 6.365×10^{-4} m/s ($N_y = 16$) à 5.487×10^{-7} m/s ($N_y = 512$), soit un ordre de Richardson $p_{L_1}, p_{L_2}, p_{L_\infty} \in [2.00, 2.07]$.
   — Vérifier que la contrainte de cisaillement reconstruite aux faces $\tau_{xz}(y_j) = \mu(y_j)\frac{u_{i,j+1}-u_{i,j}}{\Delta y}$ coïncide avec la droite exacte $-G_x(y_j - e)$ à 10^{-6} Pa près.
2. **Sensibilité paramétrique à l'exposant d'hétérogénéité a (`visc_a`) :**
   — Tester `"visc_a"` $\in \{0.0, 1.0, 2.0, 4.0\}$ (correspondant à des rapports $\mu_{\max}/\mu_{\min} \in \{1, 2.72, 7.39, 54.6\}$) et vérifier que pour $a \to 0$ on retrouve la solution parabolique exacte à la précision machine, tandis que l'ordre 2 est maintenu pour tout $a > 0$.
3. **Performance des préconditionneurs à coefficients variables et parité C++/Fortran :**
   — Vérifier que la prise en compte des viscosités variables $\mu_e, \mu_w, \mu_n, \mu_s$ dans les blocs F_u, F_v de `BLOCK_TRIANGULAR` et dans la diagonale de `JACOBI`/`ILU0` maintient une convergence rapide de `BICGSTAB2`.
   — Vérifier la parité machine $|E_{L_1}^{\text{C++}} - E_{L_1}^{\text{Fortran}}| < 10^{-15}$.

5.6.2 6.2 Résultats Attendus et Visualisation ParaView

Chapitre 6

Tutoriel 6 : Goutte Statique en Équilibre Capillaire et Loi de Young-Laplace

Ce tutoriel constitue le banc d'essai incontournable de l'hydrostatique diphasique capillaire. Il valide le modèle de force superficielle continue CSF (Continuum Surface Force), la reconstruction géométrique de courbure kappa et démontre l'absence de courants parasites excessifs autour d'une goutte circulaire à l'équilibre.

6.1 1. Objectifs Pédagogiques et Modules Logiciels Activés

6.1.1 1.1 Compétences Acquises

- Comprendre les fondements physiques et les équations aux dérivées partielles gouvernant l'écoulement.
- Configurer les paramètres numériques et conditions aux limites sur grille cartésienne décalée MAC.
- Exécuter le cas test avec les exécutables C++17 et Fortran 2018 (séquentiel et OpenMP).
- Quantifier avec rigueur la précision par rapport à la solution analytique exacte (L_2, L_∞).
- Produire et exploiter les visualisations scientifiques volumiques sous ParaView.

6.1.2 1.2 Modules Logiciels Sollicités

- **Moteur C++17 :** `Mesh2D`, `FlowFields`, `VOFTracker`, `CSFModel`, `NavierStokesSolver`, `UzawaSolver`.
- **Moteur Fortran 2018 :** `module_mesh`, `module_flow_fields`, `module_vof`, `module_csf`, `module_navier_stokes`, `module_uzawa`.

6.2 2. Formulation Physique et Solution Analytique Exacte

Équations Physiques et Configuration

Ce cas valide le couplage hydro-capillaire entre les équations de Navier-Stokes diphasiques et la force de tension superficielle *Continuum Surface Force* (CSF) discrétisée selon le schéma à équilibre exact de forces (*Balanced-Force*) sur maillage décalé MAC.

Dans un domaine carré fermé $\Omega = [0,1] \times [0,1]\,\mathrm{m}^2$ en apesanteur ($\mathbf{g} = \mathbf{0}$) avec conditions de glissement libre (`FREE_SLIP`) sur les 4 parois, une goutte circulaire de rayon $R_0 = 0.25\,\mathrm{m}$ est centrée en $(x_c, y_c) = (0.5, 0.5)\,\mathrm{m}$ au repos initial $\mathbf{u}(\mathbf{x}, 0) = \mathbf{0}$. Les deux fluides ont des propriétés identiques $\rho_1 = \rho_2 = 1.0\,\mathrm{kg/m}^3$, $\mu_1 = \mu_2 = 1.0\,\mathrm{Pa \cdot s}$, et un coefficient de tension superficielle $\sigma = 1.0\,\mathrm{N/m}$.

6.2.1 Loi analytique de Young-Laplace et régularisation CSF

À l'équilibre exact, la vitesse est identiquement nulle ($\mathbf{u}_{\mathrm{ex}} = \mathbf{0}$) et le saut de pression à la traversée de l'interface circulaire de courbure $\kappa = 1/R_0 = 4.0\,\mathrm{m}^{-1}$ vaut :

$$\Delta P_{\mathrm{ex}} = P_{\mathrm{int}} - P_{\mathrm{ext}} = \frac{\sigma}{R_0} = \frac{1.0}{0.25} = 4.0000\,\mathrm{Pa}.$$

La force capillaire volumique $\mathbf{f}_{\mathrm{st}} = \sigma\kappa\nabla C$, où $\kappa = -\nabla \cdot (\nabla\widetilde{C}/\|\nabla\widetilde{C}\|)$, est évaluée aux centres des faces de vitesse exactement aux mêmes points que $-\nabla p$. Lorsque `enable_diffusion_smoothing: true`, l'indicatrice C est préalablement régularisée par diffusion laplacienne afin d'éliminer les erreurs haute fréquence sur les dérivées secondes de la courbure et de supprimer les courants parasites (*spurious currents*).

6.3 3. Paramètres Physiques, Numériques et Discrétisation

Physiques, Numériques et Matrice des Jeux de Paramètres

Catégorie	Paramètre	Valeur / Spécification
Domaine et Goutte	Ω, (x_c, y_c), R_0	$[0,1] \times [0,1]\,\mathrm{m}^2$, $(0.5, 0.5)\,\mathrm{m}$, $R_0 = 0.25\,\mathrm{m}$
Propriétés physiques	$\rho_1 = \rho_2, \mu_1 = \mu_2, \sigma, \mathbf{g}$	$\rho = 1.0\,\mathrm{kg/m}^3$, $\mu = 1.0\,\mathrm{Pa \cdot s}$, $\sigma = 1.0\,\mathrm{N/m}$, $\mathbf{g} = \mathbf{0}$
Séquence de grilles et Δt	$N \in \{32, 64, 128, 256\}$, $t_{\mathrm{final}} = 0.05\,\mathrm{s}$	$\Delta t \in \{0.005, 0.005, 0.0025, 0.00125\}\,\mathrm{s}$
Axes d'étude	Axe A (3 VOF + lissage), Axe B (sans lissage), Axe C (solveurs)	`WY+PLIC`, `CIAM+PLIC`, `VOF_TVD` ; `AL` ($r_{\mathrm{al}} = 100$), `AAL` ($K_{\mathrm{aal}} = 100$), `MONOLITHIC` ($r_{\mathrm{al}} = 0$), `STANDARD_PROJECTION`
Solveurs et Préconditionneurs	`linear_solver` & `precond_type`	`BICGSTAB2`, `BICGSTAB`, `GMRES`, `MINRES` avec `BLOCK_TRIANGULAR`, `JACOBI`, `SSOR`, `ILU0`

6.4 4. Fichier de Configuration JSON Pas à Pas

de Fichiers `.json` (C++17 et Fortran 2018)

6.4.1 Prototype `case/cpp/cas_6_axeA_wy_plic_n128.json`

```
{
  "mode": "NAVIER_STOKES",
  "mesh": {
    "nx": 128,
    "ny": 128,
    "lx": 1.0,
    "ly": 1.0
```

```
  },
  "physics": {
    "enable_two_phase": true,
    "enable_surface_tension": true,
    "enable_convection": false,
    "enable_diffusion_smoothing": true,
    "diffusion_smoothing_steps": 3,
    "sigma": 1.0,
    "gx": 0.0,
    "gy": 0.0,
    "fluid1": { "rho": 1.0, "mu": 1.0 },
    "fluid2": { "rho": 1.0, "mu": 1.0 }
  },
  "initial_condition": {
    "type": "DROPLET",
    "center_x": 0.5,
    "center_y": 0.5,
    "radius": 0.25
  },
  "time": {
    "dt": 0.0025,
    "t_final": 0.05,
    "cfl_max": 0.3,
    "adaptive_dt": false
  },
  "boundary_conditions": {
    "left":   { "type": "FREE_SLIP", "u": 0.0, "v": 0.0 },
    "right":  { "type": "FREE_SLIP", "u": 0.0, "v": 0.0 },
    "bottom": { "type": "FREE_SLIP", "u": 0.0, "v": 0.0 },
    "top":    { "type": "FREE_SLIP", "u": 0.0, "v": 0.0 }
  },
  "solver": {
    "type": "AUGMENTED_LAGRANGIAN",
    "linear_solver": "BICGSTAB2",
    "tol": 1e-07,
    "max_iter": 300,
    "r_al": 100.0,
    "uzawa_iter": 1,
    "use_precond": true,
    "precond_type": "BLOCK_TRIANGULAR"
  },
  "vof": {
    "advection_method": "WEYMOUTH_YUE",
    "reconstruction_method": "PLIC",
    "tvd_limiter": "NONE",
    "cfl_vof": 0.5
  },
  "output": {
    "output_dir": "VALIDATION_TOTALE/validation_cas_6_laplace_droplet/output/cpp/axeA_wy_plic/n128",
    "case_name": "cas_6_axeA_wy_plic_n128",
    "vtk_interval": 2,
    "log_interval": 1,
    "write_csv_history": true
  }
}
```

6.5 5. Procédure d'Exécution (Moteurs C++17 et Fortran 2018)

L'exécution s'effectue indifféremment à l'aide du binaire C++17 (`covof2d_cpp`) ou du binaire Fortran 2018 (`covof2d_fortran`), garantissant une stricte équivalence des résultats numériques.

```
# 1. Se placer dans le répertoire du cas
cd VALIDATION_TOTALE/Run_Valide-CASES/validation_cas_6_laplace_droplet

# 2. Exécution avec le moteur C++17 (Séquentiel)
../../../COVOF_2D-C++/bin/covof2d_cpp case/cpp/cas_6_config.json

# 3. Exécution avec le moteur C++17 (Parallèle OpenMP)
export OMP_NUM_THREADS=4
../../../COVOF_2D-C++/bin/covof2d_cpp case/cpp/cas_6_config.json

# 4. Exécution avec le moteur Fortran 2018 (Séquentiel)
../../../COVOF_2D-FORTRAN/bin/covof2d_fortran case/fortran/cas_6_config.json

# 5. Exécution avec le moteur Fortran 2018 (Parallèle OpenMP)
export OMP_NUM_THREADS=4
../../../COVOF_2D-FORTRAN/bin/covof2d_fortran case/fortran/cas_6_config.json
```

6.6 6. Analyse Quantitative, Script Python et Visualisation ParaView

6.6.1 6.1 Critères de Vérification et Tolérances

à Mener (Précision, Intégrité et Performance)

1. **Saut de pression de Young-Laplace et convergence d'ordre 2 :**
   — Calculer P_{int} sur $\{r < 0.18\,\text{m}, C > 0.90\}$ et P_{ext} sur $\{r > 0.35\,\text{m}, C < 0.05\}$.
   — Vérifier que $\Delta P = P_{\text{int}} - P_{\text{ext}}$ converge de $4.0405\,\text{Pa}$ ($N = 32$, erreur $1.01\,\%$) à $3.9985\,\text{Pa}$ ($N = 128$, erreur $0.04\,\%$) et $3.9966\,\text{Pa}$ ($N = 256$, erreur $0.09\,\%$), avec un ordre de convergence d'extrapolation $|\Delta P_h - \Delta P_{2h}|$ égal à 2.00.
2. **Maîtrise des courants parasites et étude paramétrique de la régularisation CSF :**
   — Vérifier que $\|\mathbf{u}\|_\infty(N = 256) \approx 3.34 \times 10^{-3}\,\text{m/s}$ ($\text{Ca}_{\text{parasite}} = \mu\|\mathbf{u}\|_\infty/\sigma \approx 3.34 \times 10^{-3}$) avec `enable_diffusion_smoothing: true`, contre $0.190\,\text{m/s}$ sans lissage (réduction d'un facteur $60\times$).
   — Étudier la sensibilité au paramètre `diffusion_smoothing_steps` $\in \{0, 1, 2, 3, 5\}$ et au filtre `enable_svof_smoothing: true` sur l'énergie cinétique parasite $E_k(t)$ et la largeur de la zone de transition de pression.
   — Vérifier que dans ce domaine fermé avec conditions `FREE_SLIP`, la projection à moyenne nulle $\sum_c r_p[c] = 0$ dans `BlockTriangularPreconditioner` garantit l'absence totale de dérive de la pression moyenne $\int_\Omega p\,\text{d}\Omega = 0$.
3. **Invariance des solveurs et parité dual-code :**
   — Vérifier que `AL`, `AAL` et `MONOLITHIC` donnent le même ΔP à $6 \times 10^{-4}\,\text{Pa}$ près pour tol $\in \{10^{-7}, 10^{-10}, 10^{-12}\}$, et que $|p_{\text{C++}} - p_{\text{Fortran}}| \leq 1.2 \times 10^{-14}\,\text{Pa}$.

6.6.2 6.2 Résultats Attendus et Visualisation ParaView

Chapitre 7

Tutoriel 7 : Advection d'une Tâche Circulaire dans un Tourbillon Cisaillé

Ce tutoriel isole le transport cinématique pur de l'interface fluide VOF sans résolution hydrodynamique. Une tâche circulaire est soumise à un champ de vitesse cisaillé instationnaire réversible qui l'étire en une fine nappe spirale avant d'inverser le flux à t = T/2 pour reformer le disque initial à t = T.

7.1 1. Objectifs Pédagogiques et Modules Logiciels Activés

7.1.1 1.1 Compétences Acquises

- Comprendre les fondements physiques et les équations aux dérivées partielles gouvernant l'écoulement.
- Configurer les paramètres numériques et conditions aux limites sur grille cartésienne décalée MAC.
- Exécuter le cas test avec les exécutables C++17 et Fortran 2018 (séquentiel et OpenMP).
- Quantifier avec rigueur la précision par rapport à la solution analytique exacte (L_2, L_∞).
- Produire et exploiter les visualisations scientifiques volumiques sous ParaView.

7.1.2 1.2 Modules Logiciels Sollicités

- **Moteur C++17 :** `Mesh2D`, `FlowFields`, `WeymouthYueVOF`, `InterfaceReconstruction`, `VTKWriter`.
- **Moteur Fortran 2018 :** `module_mesh`, `module_flow_fields`, `module_weymouth_yue`, `module_plic_reconstruction`, `module_vtk_writer`.

7.2 2. Formulation Physique et Solution Analytique Exacte

Équations Physiques et Configuration

Ce cas cinématique valide les schémas de transport et de reconstruction d'interface Volume-of-Fluid (VOF) sous étirement tourbillonnaire sévère et réversibilité temporelle exacte.

Dans la cavité carrée unitaire $\Omega = [0,1] \times [0,1]\,\mathrm{m}^2$, un disque fluide initial ($C = 1$) de rayon $R_0 = 0.125\,\mathrm{m}$ est centré en $(x_c, y_c) = (0.5, 0.75)\,\mathrm{m}$ à $t = 0$. L'indicatrice de phase $C(\mathbf{x}, t) \in [0,1]$ obéit à l'équation d'advection pure sans terme source :

$$\frac{\partial C}{\partial t} + \nabla \cdot (C\mathbf{u}) = 0.$$

Le champ de vitesse solénoïdal ($\nabla \cdot \mathbf{u} \equiv 0$) dérive de la fonction de courant non stationnaire modulée par un cosinus temporel de période $T = 6.0\,\mathrm{s}$:

$$\Psi(x, y, t) = \frac{1}{\pi} \sin^2(\pi x) \sin^2(\pi y) \cos\left(\frac{\pi t}{T}\right),$$

soit aux faces des mailles décalées MAC :

$$u(x, y, t) = -\frac{\partial \Psi}{\partial y} = -\sin^2(\pi x) \sin(2\pi y) \cos\left(\frac{\pi t}{T}\right),$$

$$v(x, y, t) = +\frac{\partial \Psi}{\partial x} = +\sin(2\pi x) \sin^2(\pi y) \cos\left(\frac{\pi t}{T}\right).$$

De $t = 0$ à $t = T/2 = 3.0\,\mathrm{s}$, le disque est étiré en une fine spirale tourbillonnaire ; à $t = 3.0\,\mathrm{s}$, le champ de vitesse change de signe et réenroule le filament jusqu'à $t = T = 6.0\,\mathrm{s}$ où la solution exacte coïncide identiquement avec l'état initial :

$$C_{\mathrm{ex}}(\mathbf{x}, T) \equiv C(\mathbf{x}, 0).$$

7.3 3. Paramètres Physiques, Numériques et Discrétisation

Physiques, Numériques et Matrice des Jeux de Paramètres

Catégorie	Paramètre	Valeur / Spécification
Domaine et Disque initial	Ω, (x_c, y_c), R_0	$[0,1] \times [0,1]\,\mathrm{m}^2$, $(x_c, y_c) = (0.5, 0.75)\,\mathrm{m}$, $R_0 = 0.125\,\mathrm{m}$
Période cinématique	$T = t_{\mathrm{final}}$	$6.0\,\mathrm{s}$ (inversion à $T/2 = 3.0\,\mathrm{s}$)
Séquence de grilles et Δt	$(N \times N, \Delta t)$ (CFL ≤ 0.32)	$(64^2, 0.005\,\mathrm{s})$, $(128^2, 0.0025\,\mathrm{s})$, $(256^2, 0.00125\,\mathrm{s})$, $(512^2, 0.0005\,\mathrm{s})$
Méthodes d'advection VOF	`vof.advection_method`	1) `"WEYMOUTH_YUE"`, 2) `"CIAM"`, 3) `"VOF_TVD"`
Reconstructions et limiteurs	`reconstruction_method` / `tvd_limiter`	`"PLIC"`, `"ELVIRA"`, `"LVIRA"`, `"SLIC"` ; `"SUPERBEE"`, `"VAN_LEER"`, `"MUSCL"`, `"HRIC"`
Couplage Solveurs (Phase 2)	`solver.type` & `precond_type`	`AL+DIRECT`, `AL+BICGSTAB2+BT`, `AAL+BICGSTAB2+BT`, `MONOLITHIC+BICGSTAB2+BT`

7.4 4. Fichier de Configuration JSON Pas à Pas

de Fichiers `.json` (C++17 et Fortran 2018)

7.4.1 Prototype `case/cpp/cas_7_weymouth_yue_plic_n128.json`

```
{
  "mode": "VORTEX_ADVECTION",
  "mesh": {
    "nx": 128,
    "ny": 128,
    "lx": 1.0,
    "ly": 1.0
```

```
  },
  "physics": {
    "enable_two_phase": true,
    "enable_surface_tension": false,
    "enable_convection": true,
    "sigma": 0.0,
    "fluid1": { "rho": 1000.0, "mu": 0.001 },
    "fluid2": { "rho": 1.0,    "mu": 1.0e-05 }
  },
  "vof": {
    "advection_method": "WEYMOUTH_YUE",
    "reconstruction_method": "PLIC",
    "tvd_limiter": "NONE",
    "cfl_vof": 0.5
  },
  "initial_condition": {
    "type": "VORTEX_SHEAR",
    "center_x": 0.5,
    "center_y": 0.75,
    "radius": 0.125
  },
  "time": {
    "dt": 0.0025,
    "t_final": 6.0,
    "adaptive_dt": false
  },
  "boundary_conditions": {
    "left":   { "type": "DIRICHLET", "u": 0.0, "v": 0.0 },
    "right":  { "type": "DIRICHLET", "u": 0.0, "v": 0.0 },
    "bottom": { "type": "DIRICHLET", "u": 0.0, "v": 0.0 },
    "top":    { "type": "DIRICHLET", "u": 0.0, "v": 0.0 }
  },
  "solver": {
    "type": "AUGMENTED_LAGRANGIAN",
    "linear_solver": "DIRECT",
    "tol": 1e-12,
    "max_iter": 500,
    "r_al": 100.0,
    "uzawa_iter": 1,
    "use_precond": false,
    "precond_type": "NONE"
  },
  "output": {
    "output_dir": "VALIDATION_TOTALE/validation_cas_7_vortex_shear/output/cpp/weymouth_yue_plic/n128",
    "case_name": "cas_7_weymouth_yue_plic_n128",
    "vtk_interval": 240,
    "log_interval": 200,
    "write_csv_history": true
  }
}
```

7.5 5. Procédure d'Exécution (Moteurs C++17 et Fortran 2018)

L'exécution s'effectue indifféremment à l'aide du binaire C++17 (`covof2d_cpp`) ou du binaire Fortran 2018 (`covof2d_fortran`), garantissant une stricte équivalence des résultats numériques.

```
# 1. Se placer dans le répertoire du cas
cd VALIDATION_TOTALE/Run_Valide-CASES/validation_cas_7_vortex_shear

# 2. Exécution avec le moteur C++17 (Séquentiel)
../../../COVOF_2D-C++/bin/covof2d_cpp case/cpp/cas_7_config.json
```

```
# 3. Exécution avec le moteur C++17 (Parallèle OpenMP)
export OMP_NUM_THREADS=4
../../../COVOF_2D-C++/bin/covof2d_cpp case/cpp/cas_7_config.json

# 4. Exécution avec le moteur Fortran 2018 (Séquentiel)
../../../COVOF_2D-FORTRAN/bin/covof2d_fortran case/fortran/cas_7_config.json

# 5. Exécution avec le moteur Fortran 2018 (Parallèle OpenMP)
export OMP_NUM_THREADS=4
../../../COVOF_2D-FORTRAN/bin/covof2d_fortran case/fortran/cas_7_config.json
```

7.6 6. Analyse Quantitative, Script Python et Visualisation ParaView

7.6.1 6.1 Critères de Vérification et Tolérances

à Mener (Précision, Intégrité et Performance)

1. **Erreur géométrique de retour L_1 à $t = T = 6.0\,\mathrm{s}$ et ordre de convergence :**

$$E_{L_1} = \frac{1}{N^2} \sum_{i=1}^{N} \sum_{j=1}^{N} |C_{i,j}(T) - C_{i,j}(0)|.$$

   Vérifier que `WEYMOUTH_YUE + PLIC` décroît de 7.35×10^{-3} ($N = 64$) à 2.32×10^{-3} ($N = 128$, $p = 1.66$), 9.42×10^{-4} ($N = 256$, $p = 1.30$) et 4.29×10^{-4} ($N = 512$, $p = 1.14$), sans aucune saturation sur grille fine.
2. **Conservation de la masse, bornitude et exploration des options VOF du code :**
   — Vérifier que $\frac{\sum C_{i,j}(t)}{\sum C_{i,j}(0)} = 100.0000\,\%$ pour `WEYMOUTH_YUE` et `VOF_TVD` (et $> 98.9\,\%$ sur grille fine pour `CIAM`), et que $0 \leq C_{i,j}(t) \leq 1$ en tout point et à tout instant.
   — Vérifier la règle d'exclusion mutuelle implémentée dans `Config.cpp` / `module_config.f90` : lorsque `advection_method = "VOF_TVD"`, `reconstruction_method` est automatiquement forcé à `"NONE"` et le limiteur par défaut à `"SUPERBEE"` ; lorsque `advection_method` est géométrique (`"WEYMOUTH_YUE"` ou `"CIAM"`), `tvd_limiter` est forcé à `"NONE"`.
   — Comparer la netteté de l'interface et l'erreur E_{L_1} entre les limiteurs algébriques `SUPERBEE`, `VAN_LEER`, `MUSCL` et `HRIC`, ainsi qu'entre les reconstructions `PLIC`, `ELVIRA`, `LVIRA` et `SLIC`.
3. **Invariance vis-à-vis du solveur de couplage et parité C++17 / Fortran 2018 :**
   — Vérifier en Phase 2 que les erreurs $E_{L_1}, E_{L_2}, E_{L_\infty}$ sont rigoureusement identiques entre `AL+DIRECT`, `AL+BICGSTAB2+BLOCK_TRIANGULAR`, `AAL+BICGSTAB2+BLOCK_TRIANGULAR` et `MONOLITHIC+BICGSTAB2+BLOCK_TRIANGULAR`.
   — Vérifier la parité machine C++17 vs Fortran 2018 : $|E_{L_1}^{\mathrm{C++}} - E_{L_1}^{\mathrm{Fortran}}| < 10^{-14}$.

7.6.2 6.2 Résultats Attendus et Visualisation ParaView

Chapitre 8

Tutoriel 8 : Chute Libre d'un Cylindre Dense dans le Vide (Pénalisation de Brinkman)

Ce tutoriel valide le couplage fluide-solide par pénalisation de Brinkman avec un contraste de masse volumique extrême (rapport rho_s / rho_f = 85 000). Il démontre que l'inclusion solide suit rigoureusement la cinématique galiléenne de chute libre dans le vide sans dispersion ni amortissement numérique parasite.

8.1 1. Objectifs Pédagogiques et Modules Logiciels Activés

8.1.1 1.1 Compétences Acquises

- Comprendre les fondements physiques et les équations aux dérivées partielles gouvernant l'écoulement.
- Configurer les paramètres numériques et conditions aux limites sur grille cartésienne décalée MAC.
- Exécuter le cas test avec les exécutables C++17 et Fortran 2018 (séquentiel et OpenMP).
- Quantifier avec rigueur la précision par rapport à la solution analytique exacte (L_2, L_∞).
- Produire et exploiter les visualisations scientifiques volumiques sous ParaView.

8.1.2 1.2 Modules Logiciels Sollicités

- **Moteur C++17 :** `Mesh2D`, `FlowFields`, `BrinkmanPenalization`, `NavierStokesSolver`, `BiCGSTAB2Solver`, `SSORPreconditioner`.
- **Moteur Fortran 2018 :** `module_mesh`, `module_flow_fields`, `module_brinkman`, `module_navier_stokes`, `module_linear_solvers`, `module_preconditioners`.

8.2 2. Formulation Physique et Solution Analytique Exacte

Équations Physiques et Configuration

Ce cas diphasique sévère simule la chute libre sous gravité d'un cylindre circulaire ultra-dense et ultra-visqueux dans l'air au repos. Il met à l'épreuve la robustesse des solveurs de couplage vitesse-pression et des préconditionneurs face à des sauts extrêmes de masse volumique $r_\rho = \rho_2/\rho_1 \approx 84\,976$ et de viscosité dynamique $r_\mu = \mu_2/\mu_1 \approx 5.405 \times 10^9$.

Dans une cavité rectangulaire $\Omega = [0, 0.1] \times [0, 0.2]\,\mathrm{m}^2$ soumise à la pesanteur $\mathbf{g} = (0, -9.81)\,\mathrm{m/s}^2$, un cylindre de rayon $R = 0.0125\,\mathrm{m}$ est initialement centré en $(x_c, Y_0) = (0.05, 0.15)\,\mathrm{m}$ sans vitesse initiale : - **Fluide 1 (Air ambiant, $C = 0$)** : $\rho_1 = 1.1768\,\mathrm{kg/m}^3$, $\mu_1 = 1.85 \times 10^{-5}\,\mathrm{Pa \cdot s}$. - **Fluide 2 (Cylindre dense, $C = 1$)** : $\rho_2 = 100\,000\,\mathrm{kg/m}^3$, $\mu_2 = 100\,000\,\mathrm{Pa \cdot s}$, $\sigma = 0$.

Compte tenu du rapport d'inertie $r_\rho \gg 1$, la traînée aérodynamique et la poussée d'Archimède exercées par l'air sont négligeables devant le poids du cylindre. La vitesse verticale $U_c(t)$ et l'ordonnée du centre de gravité $Y_c(t)$ suivent la loi de chute libre galiléenne exacte dans le vide :

$$U_c^{\mathrm{ex}}(t) = -gt, \qquad Y_c^{\mathrm{ex}}(t) = Y_0 - \frac{1}{2}gt^2.$$

À l'instant final $t_{\mathrm{final}} = 0.144\,\mathrm{s}$:

$$U_c^{\mathrm{ex}}(0.144\,\mathrm{s}) = -1.41264\,\mathrm{m/s}, \qquad Y_c^{\mathrm{ex}}(0.144\,\mathrm{s}) = 0.048288\,\mathrm{m}.$$

8.2.1 Formulations numériques spécifiques mises en œuvre

1. **Schéma temporel Gear / BDF2 d'ordre 2** : $\frac{1.5\rho^{n+1}\mathbf{u}^{n+1} - 2\rho^n\mathbf{u}^n + 0.5\rho^{n-1}\mathbf{u}^{n-1}}{\Delta t}$ (`"time_scheme": "GEAR"`).
2. **Linéarisation convective d'Adams-Bashforth** : vitesse advectrice extrapolée $\mathbf{u}^* = 2\mathbf{u}^n - \mathbf{u}^{n-1}$ (`"convective_linearization": "ADAMS_BASHFORTH"`).
3. **Lissage SVOF de Helmholtz** : $(I - D\Delta)C_s = C$ avec $D = k\Delta x\Delta y$ ($k = 1.0$) et moyenne harmonique nodale de la viscosité de cisaillement $\mu_{\mathrm{node}} = \frac{\mu_1\mu_2}{\mu_2\widetilde{C}_{s,\mathrm{node}} + \mu_1(1-\widetilde{C}_{s,\mathrm{node}})}$.
4. **Quatre méthodes de couplage vitesse-pression comparées** :
   — `MONOLITHIC` (**FC**, $r_{\mathrm{al}} = 0$) : résolution couplée directe du système selle par BiCGSTAB(2) + préconditionneur `BLOCK_TRIANGULAR`.
   — `AUGMENTED_LAGRANGIAN` (**AL**, $r_{\mathrm{al}} = 10\,000$ fixe).
   — `ADAPTATIVE_AUGMENTED_LAGRANGIAN` (**AAL**) : paramètre local par cellule $r(t, M)$ adapté selon les gradients d'interface et la divergence locale avec facteur K_{aal} (`k_aal`) et critère d'arrêt d'Uzawa `uzawa_tol`.
   — `STANDARD_PROJECTION` (**SP**) : méthode de projection à pas fractionnaire.

8.3 3. Paramètres Physiques, Numériques et Discrétisation

Physiques, Numériques et Matrice des Jeux de Paramètres

Catégorie	Paramètre	Valeur / Spécification
Domaine et Cylindre	Ω, (x_c, Y_0), R	$[0, 0.1] \times [0, 0.2]\,\mathrm{m}^2$, $(0.05, 0.15)\,\mathrm{m}$, $R = 0.0125\,\mathrm{m}$
Maillages et Pas de temps	$(N_x \times N_y)$, Δt, t_{final}	100×200 et 200×400, $\Delta t = 6.25 \times 10^{-5}\,\mathrm{s}$, $t_{\mathrm{final}} = 0.144\,\mathrm{s}$ (2304 pas)
Conditions aux limites	Gauche/Droite ; Bas/Haut	`FREE_SLIP` sur $x = 0, 0.1$; `DIRICHLET` ($u = v = 0$) sur $y = 0, 0.2$
Solveurs et Préconditionneurs	Couplage, Krylov, Précond.	`MONOLITHIC` (FC), `AUGMENTED_LAGRANGIAN` (AL), `AAL` (`k_aal` $\in \{10, 100, 1000\}$), `STANDARD_PROJECTION` (SP) ; `BICGSTAB2`, `GMRES`, `BICGSTAB` avec `BLOCK_TRIANGULAR`, `JACOBI`, `SSOR`, `ILU0`

8.4 4. Fichier de Configuration JSON Pas à Pas

de Fichiers `.json` (C++17 et Fortran 2018)

8.4.1 Prototype `case/cpp/cas_8_fc_100x200.json`

```
{
  "mode": "NAVIER_STOKES",
  "mesh": {
    "nx": 100,
    "ny": 200,
    "lx": 0.1,
    "ly": 0.2
  },
  "physics": {
    "enable_two_phase": true,
    "enable_surface_tension": false,
    "enable_convection": true,
    "convective_linearization": "ADAMS_BASHFORTH",
    "enable_svof_smoothing": true,
    "svof_filter_width": 1.0,
    "enable_diffusion_smoothing": false,
    "sigma": 0.0,
    "gx": 0.0,
    "gy": -9.81,
    "fluid1": { "rho": 1.1768,   "mu": 1.85e-05 },
    "fluid2": { "rho": 100000.0, "mu": 100000.0 }
  },
  "initial_condition": {
    "type": "CYLINDER",
    "center_x": 0.05,
    "center_y": 0.15,
    "radius": 0.0125
  },
  "time": {
    "time_scheme": "GEAR",
    "dt": 6.25e-05,
    "t_final": 0.144,
    "cfl_max": 0.5,
    "adaptive_dt": false
  },
  "boundary_conditions": {
    "left":   { "type": "FREE_SLIP", "u": 0.0, "v": 0.0 },
    "right":  { "type": "FREE_SLIP", "u": 0.0, "v": 0.0 },
    "bottom": { "type": "DIRICHLET", "u": 0.0, "v": 0.0 },
    "top":    { "type": "DIRICHLET", "u": 0.0, "v": 0.0 }
  },
  "solver": {
    "type": "MONOLITHIC",
    "linear_solver": "BICGSTAB2",
    "tol": 1e-06,
    "max_iter": 120,
    "r_al": 0.0,
    "uzawa_iter": 1,
    "use_precond": true,
    "precond_type": "BLOCK_TRIANGULAR"
  },
  "vof": {
    "advection_method": "WEYMOUTH_YUE",
    "reconstruction_method": "PLIC",
    "tvd_limiter": "NONE",
    "cfl_vof": 0.5
  },
  "output": {
    "output_dir": "VALIDATION_TOTALE/validation_cas_8_dense_cylinder/output/cpp/fc_100x200",
    "case_name": "cas_8_fc_100x200",
```

```
    "vtk_interval": 100,
    "log_interval": 100,
    "write_csv_history": true
  }
}
```

Pour activer le Lagrangien Augmenté Adaptatif (**AAL**), utiliser le bloc `"solver"` suivant :

```
  "solver": {
    "type": "ADAPTATIVE_AUGMENTED_LAGRANGIAN",
    "linear_solver": "BICGSTAB2",
    "tol": 1e-06,
    "max_iter": 200,
    "r_al": 0.0,
    "adaptative_al": true,
    "k_aal": 100.0,
    "uzawa_iter": 5,
    "uzawa_tol": 1e-06,
    "use_precond": true,
    "precond_type": "BLOCK_TRIANGULAR"
  }
```

8.5 5. Procédure d'Exécution (Moteurs C++17 et Fortran 2018)

L'exécution s'effectue indifféremment à l'aide du binaire C++17 (`covof2d_cpp`) ou du binaire Fortran 2018 (`covof2d_fortran`), garantissant une stricte équivalence des résultats numériques.

```
# 1. Se placer dans le répertoire du cas
cd VALIDATION_TOTALE/Run_Valide-CASES/validation_cas_8_dense_cylinder

# 2. Exécution avec le moteur C++17 (Séquentiel)
../../../COVOF_2D-C++/bin/covof2d_cpp case/cpp/cas_8_config.json

# 3. Exécution avec le moteur C++17 (Parallèle OpenMP)
export OMP_NUM_THREADS=4
../../../COVOF_2D-C++/bin/covof2d_cpp case/cpp/cas_8_config.json

# 4. Exécution avec le moteur Fortran 2018 (Séquentiel)
../../../COVOF_2D-FORTRAN/bin/covof2d_fortran case/fortran/cas_8_config.json

# 5. Exécution avec le moteur Fortran 2018 (Parallèle OpenMP)
export OMP_NUM_THREADS=4
../../../COVOF_2D-FORTRAN/bin/covof2d_fortran case/fortran/cas_8_config.json
```

8.6 6. Analyse Quantitative, Script Python et Visualisation ParaView

8.6.1 6.1 Critères de Vérification et Tolérances

à Mener (Précision, Intégrité et Performance)

1. **Erreurs relatives L_2 temporelles sur la vitesse et la position :**

$$\mathrm{Err}_{\mathrm{rel},2}(U_c) = \sqrt{\frac{\sum_{n=1}^{N_t} \left(U_c(t^n) - U_c^{\mathrm{ex}}(t^n)\right)^2}{\sum_{n=1}^{N_t} \left(U_c^{\mathrm{ex}}(t^n)\right)^2}}, \qquad \mathrm{Err}_{\mathrm{rel},2}(Y_c) = \sqrt{\frac{\sum_{n=1}^{N_t} \left(Y_c(t^n) - Y_c^{\mathrm{ex}}(t^n)\right)^2}{\sum_{n=1}^{N_t} \left(Y_c^{\mathrm{ex}}(t^n)\right)^2}}.$$

Sur grille 100×200, vérifier que `FC` atteint $\mathrm{Err}_{\mathrm{rel},2}(U_c)\approx 8.71\times10^{-5}$ et $\mathrm{Err}_{\mathrm{rel},2}(Y_c)\approx 1.57\times10^{-4}$, avec une variation relative de volume $\Delta V/V_0 < 0.001\,\%$.

2. **Intégrité des méthodes de couplage et du filtre SVOF face aux forts contrastes ($r_\rho \approx 85\,000, r_\mu \approx 5.4\times10^9$) :**
   — Comparer `MONOLITHIC` (FC), `AUGMENTED_LAGRANGIAN` ($r_{\mathrm{al}} = 10^4$), `ADAPTATIVE_AUGMENTED_LAGRANGIAN` (`AAL` avec $K_{\mathrm{aal}} \in \{10, 100, 1000\}$) et `STANDARD_PROJECTION` (SP) sur la préservation de la circularité du cylindre et l'absence de traînée numérique parasite.
   — Vérifier l'impact du filtre elliptique de Helmholtz (`enable_svof_smoothing: true` vs `false`, `svof_filter_width` $k \in \{0.5, 1.0, 2.0\}$) et de la moyenne harmonique nodale de μ sur l'élimination des pics de vorticité non physiques à l'interface.
3. **Performance des préconditionneurs à saut de densité et parité C++/Fortran :**
   — Vérifier que le complément de Schur à densité variable $a_W = \Delta y^2/D_u$, $a_E = \Delta y^2/D_u$, $a_S = \Delta x^2/D_v$, $a_N = \Delta x^2/D_v$ dans `BlockTriangularPreconditioner` (utilisant $\alpha_{\mathrm{time}} = 1.5$ en mode `GEAR`) maintient la convergence de `BICGSTAB2` en < 120 itérations malgré le saut de densité $1:85\,000$.
   — Comparer le temps CPU total et la divergence résiduelle $\|\nabla\cdot\mathbf{u}\|_\infty$ entre C++17 et Fortran 2018.

8.6.2 6.2 Résultats Attendus et Visualisation ParaView

Chapitre 9

Tutoriel 9 : Sédimentation Confinée 2D d'une Inclusion Cylindrique et Loi de Faxén

Ce tutoriel modélise la sédimentation d'un cylindre solide en régime rampant de Stokes (Re « 1) entre deux parois planes verticales. Il confronte la vitesse limite de chute numérique stationnaire avec la loi analytique de Faxén intégrant la correction de confinement pariétal.

9.1 1. Objectifs Pédagogiques et Modules Logiciels Activés

9.1.1 1.1 Compétences Acquises

- Comprendre les fondements physiques et les équations aux dérivées partielles gouvernant l'écoulement.
- Configurer les paramètres numériques et conditions aux limites sur grille cartésienne décalée MAC.
- Exécuter le cas test avec les exécutables C++17 et Fortran 2018 (séquentiel et OpenMP).
- Quantifier avec rigueur la précision par rapport à la solution analytique exacte (L_2, L_∞).
- Produire et exploiter les visualisations scientifiques volumiques sous ParaView.

9.1.2 1.2 Modules Logiciels Sollicités

- **Moteur C++17 :** `Mesh2D`, `FlowFields`, `BrinkmanPenalization`, `StokesSolver`, `BiCGSTAB2Solver`, `SSORPreconditioner`.
- **Moteur Fortran 2018 :** `module_mesh`, `module_flow_fields`, `module_brinkman`, `module_stokes`, `module_linear_solvers`, `module_preconditioners`.

9.2 2. Formulation Physique et Solution Analytique Exacte

Équations Physiques et Configuration

Ce vingt-deuxième cas valide la méthode de pénalisation de viscosité combinée au Lagrangien Augmenté Adaptatif (`ADAPTATIVE_AUGMENTED_LAGRANGIAN`) pour la sédimentation lente d'un cylindre rigide confiné entre deux parois verticales, comparée à la loi analytique de Faxén ($Re_p \ll 1, Ga < 1$).

Dans une cavité carrée fermée $\Omega = [0, 0.1] \times [0, 0.1]\,\mathrm{m}^2$ ($2l = 0.1\,\mathrm{m}$, $l = 0.05\,\mathrm{m}$) à parois adhérentes ($\mathbf{u} = \mathbf{0}$), un cylindre de rayon $a = 0.00625\,\mathrm{m}$ (rapport de confinement $k_x = a/l = 1/8 = 0.125$) est

centré en $(0.05, 0.05)\,\mathrm{m}$. Le fluide porteur ($C = 0$) a $\rho_f = 1000\,\mathrm{kg/m^3}$, $\mu_f = 0.153125\,\mathrm{Pa\cdot s}$, et la particule ($C = 1$) a $\rho_p = 1500\,\mathrm{kg/m^3}$, $\mu_p = 1000\,\mathrm{Pa\cdot s}$ sous $g_y = -9.81\,\mathrm{m/s^2}$.

9.2.0.1 Loi analytique de vitesse terminale de Faxén

En régime de Stokes 2D confiné ($k_x = a/l = 0.125$), la vitesse terminale analytique vaut :

$$U_{c,\mathrm{th}} = \frac{(\rho_f - \rho_p)\, g\, a^2}{4\,\mu_f}\,(-0.9157 - \ln(k_x) + 1.7244\,k_x^2 - 1.7302\,k_x^4) = -0.05060\,\mathrm{m/s},$$

$$E_{c,\mathrm{th}} \approx 2.7554 \times 10^{-3}\,\mathrm{J}.$$

9.3 3. Paramètres Physiques, Numériques et Discrétisation

Physiques, Numériques et Matrice des Jeux de Paramètres

Catégorie	Paramètre	Valeur / Spécification
Domaine et confinement k_x	$[0, 2l]^2$, a, $k_x = a/l$	$2l = 0.1\,\mathrm{m}$ ($l = 0.05\,\mathrm{m}$), $a = 0.00625\,\mathrm{m}$, $k_x = 1/8 = 0.125$, centre $(0.05, 0.05)\,\mathrm{m}$
Particule ($C = 1$) et Fluide ($C = 0$)	ρ_p, μ_p ; ρ_f, μ_f	$\rho_1 = \rho_p = 1500\,\mathrm{kg/m^3}$, $\mu_1 = 1000\,\mathrm{Pa\cdot s}$; $\rho_2 = \rho_f = 1000\,\mathrm{kg/m^3}$, $\mu_2 = 0.153125\,\mathrm{Pa\cdot s}$
Séquence de grilles et Δt	64×128, 128×128, 256×256	$\Delta t = 5 \times 10^{-4}\,\mathrm{s}$, $t_{\mathrm{final}} = 0.15\,\mathrm{s}$ (atteinte du plateau quasi-stationnaire dès $t \approx 0.04\,\mathrm{s}$)
Solveurs et filtrage SVOF	`solver` & `svof`	`ADAPTATIVE_AUGMENTED_LAGRANGIAN` (`k_aal`=100), `AUGMENTED_LAGRANGIAN`, `MONOLITHIC` ; `enable_svof_smoothing: true`

9.4 4. Fichier de Configuration JSON Pas à Pas

de Fichiers `.json` (C++17 et Fortran 2018)

9.4.1 Prototype `case/cpp/cas_9_sedimentation_faxen_n128.json`

(Pour le moteur Fortran, remplacer ***`output/cpp/`*** *par* ***`output/fortran/`*** *dans* ***`output_dir`****)*

```
{
  "mode": "NAVIER_STOKES",
  "mesh": {
    "nx": 128,
    "ny": 128,
    "lx": 0.1,
    "ly": 0.1
  },
  "physics": {
    "enable_two_phase": true,
    "enable_convection": true,
    "convective_scheme": "CENTERED",
    "convective_linearization": "PICARD",
    "enable_surface_tension": false,
```

```
    "enable_svof_smoothing": true,
    "svof_filter_width": 0.5,
    "gx": 0.0,
    "gy": -9.81,
    "fluid1": { "rho": 1500.0, "mu": 1000.0 },
    "fluid2": { "rho": 1000.0, "mu": 0.153125 }
  },
  "initial_condition": {
    "type": "DROPLET",
    "center_x": 0.05,
    "center_y": 0.05,
    "radius": 0.00625
  },
  "time": {
    "dt": 0.0005,
    "t_final": 0.15,
    "time_scheme": "GEAR",
    "adaptive_dt": false
  },
  "boundary_conditions": {
    "left":   { "type": "DIRICHLET", "u": 0.0, "v": 0.0 },
    "right":  { "type": "DIRICHLET", "u": 0.0, "v": 0.0 },
    "bottom": { "type": "DIRICHLET", "u": 0.0, "v": 0.0 },
    "top":    { "type": "DIRICHLET", "u": 0.0, "v": 0.0 }
  },
  "solver": {
    "type": "ADAPTATIVE_AUGMENTED_LAGRANGIAN",
    "adaptative_al": true,
    "k_aal": 100.0,
    "uzawa_tol": 1e-7,
    "uzawa_iter": 6,
    "linear_solver": "BICGSTAB2",
    "tol": 1e-08,
    "max_iter": 800,
    "use_precond": true,
    "precond_type": "BLOCK_TRIANGULAR"
  },
  "vof": {
    "advection_method": "WEYMOUTH_YUE",
    "reconstruction_method": "PLIC",
    "tvd_limiter": "NONE",
    "cfl_vof": 0.3
  },
  "output": {
    "output_dir": "VALIDATION_TOTALE/validation_cas_9_sedimentation_faxen/output/cpp/n128",
    "case_name": "cas_9_sedimentation_faxen_n128",
    "vtk_interval": 30,
    "log_interval": 10,
    "write_csv_history": true
  }
}
```

9.5 5. Procédure d'Exécution (Moteurs C++17 et Fortran 2018)

L'exécution s'effectue indifféremment à l'aide du binaire C++17 (`covof2d_cpp`) ou du binaire Fortran 2018 (`covof2d_fortran`), garantissant une stricte équivalence des résultats numériques.

```
# 1. Se placer dans le répertoire du cas
cd VALIDATION_TOTALE/Run_Valide-CASES/validation_cas_9_sedimentation_faxen

# 2. Exécution avec le moteur C++17 (Séquentiel)
```

```
../../../COVOF_2D-C++/bin/covof2d_cpp case/cpp/cas_9_config.json

# 3. Exécution avec le moteur C++17 (Parallèle OpenMP)
export OMP_NUM_THREADS=4
../../../COVOF_2D-C++/bin/covof2d_cpp case/cpp/cas_9_config.json

# 4. Exécution avec le moteur Fortran 2018 (Séquentiel)
../../../COVOF_2D-FORTRAN/bin/covof2d_fortran case/fortran/cas_9_config.json

# 5. Exécution avec le moteur Fortran 2018 (Parallèle OpenMP)
export OMP_NUM_THREADS=4
../../../COVOF_2D-FORTRAN/bin/covof2d_fortran case/fortran/cas_9_config.json
```

9.6 6. Analyse Quantitative, Script Python et Visualisation ParaView

9.6.1 6.1 Critères de Vérification et Tolérances

à Mener (Précision, Intégrité et Performance)

1. **Précision de la vitesse terminale vis-à-vis de la loi analytique de Faxén ($U_{c,\mathrm{th}} = -0.05060\,\mathrm{m/s}$) :** Extraire $V_{\mathrm{drop}}(t)$ et $E_k(t)$ sur $64 \times 128, 128 \times 128, 256 \times 256$ et vérifier que le plateau de vitesse concorde à moins de $3.5\,\%$ avec $-0.05060\,\mathrm{m/s}$ ($E_{k,\max} \approx 2.755 \times 10^{-3}\,\mathrm{J}$).
2. **Indéformabilité du cylindre ($\mu_p/\mu_f \approx 6530$) et rôle de `AAL` :** Vérifier que la pénalité adaptative $r_{i,j}(t)$ verrouille l'incompressibilité et la rigidité sur l'interface du cylindre tout en accélérant `BICGSTAB2`.
3. **Parité C++17 / Fortran 2018 :** Vérifier l'identité des courbes $V_{\mathrm{drop}}(t)$ et $E_k(t)$ entre C++ et Fortran.

9.6.2 6.2 Résultats Attendus et Visualisation ParaView

Chapitre 10

Tutoriel 10 : Compression d'Air par Injection d'Eau (Piston Compressible Bas Mach)

Ce tutoriel valide la méthode du Lagrangien Augmenté Compressible (C-AAL) sur un cas d'injection d'un liquide incompressible comprimant une poche de gaz à bas nombre de Mach. La montée en pression est comparée à la loi analytique isotherme/adiabatique de compression de gaz parfait.

10.1 1. Objectifs Pédagogiques et Modules Logiciels Activés

10.1.1 1.1 Compétences Acquises

— Comprendre les fondements physiques et les équations aux dérivées partielles gouvernant l'écoulement.
— Configurer les paramètres numériques et conditions aux limites sur grille cartésienne décalée MAC.
— Exécuter le cas test avec les exécutables C++17 et Fortran 2018 (séquentiel et OpenMP).
— Quantifier avec rigueur la précision par rapport à la solution analytique exacte (L_2, L_∞).
— Produire et exploiter les visualisations scientifiques volumiques sous ParaView.

10.1.2 1.2 Modules Logiciels Sollicités

— **Moteur C++17 :** `Mesh2D`, `FlowFields`, `CAALSolver`, `CompressibleGasEOS`, `UzawaPressureUpdate`.
— **Moteur Fortran 2018 :** `module_mesh`, `module_flow_fields`, `module_caal`, `module_gas_eos`, `module_uzawa`.

10.2 2. Formulation Physique et Solution Analytique Exacte

Équations Physiques et Configuration

Ce onzième cas valide l'extension du modèle diphasique 1-fluide aux écoulements faiblement compressibles à bas nombre de Mach (`compressible: true`), dans lesquels un liquide quasi-incompressible comprime un gaz compressible au sein d'une enceinte étanche.

Le système gouvernant couple l'équation de continuité compressible isotherme $\chi_T(C) = C\,\chi_{T,1} + (1 - C)\,\chi_{T,2}$, l'équation de quantité de mouvement et l'advection de la fonction couleur avec terme de

dilatation $C\,\nabla\cdot\mathbf{u}$:

$$\chi_T(C)\frac{\partial p}{\partial t}+\nabla\cdot\mathbf{u}=0,\qquad \rho(C)\left(\frac{\partial\mathbf{u}}{\partial t}+(\mathbf{u}\cdot\nabla)\mathbf{u}\right)=-\nabla p+\nabla\cdot\left[\mu(C)\left(\nabla\mathbf{u}+\nabla\mathbf{u}^T-\frac{2}{3}(\nabla\cdot\mathbf{u})\mathbf{I}\right)\right],$$

$$\frac{\partial C}{\partial t}+\nabla\cdot(\mathbf{u}C)=C\,\nabla\cdot\mathbf{u},\qquad \rho_i(p)=\rho_{i,0}\left[1+\chi_{T,i}\,(p-p_0)\right].$$

10.2.0.1 Configuration géométrique et loi analytique exacte

Dans une cavité carrée $\Omega=[0,0.1]\times[0,0.1]\,\mathrm{m}^2$ sans gravité ($\mathbf{g}=\mathbf{0}$) ni capillarité ($\sigma=0$), l'eau ($C=1$) occupe initialement $y\in[0,0.025]\,\mathrm{m}$, surmontée d'une hauteur $H_0=0.075\,\mathrm{m}$ d'air ($C=0$) à $p_0=101\,325\,\mathrm{Pa}$. De l'eau est injectée par la face inférieure $y=0$ à la vitesse verticale constante $V_0=0.1\,\mathrm{m/s}$, les trois autres parois étant à glissement libre (`FREE_SLIP`). L'interface plane monte selon $y_{\mathrm{int}}(t)=0.025+V_0t$, et la masse volumique ainsi que la pression de l'air obéissent à la loi exacte :

$$\rho_{\mathrm{air}}(t)=\frac{\rho_{\mathrm{air},0}}{1-\dfrac{V_0\,t}{H_0}},\qquad p(t)=\frac{p_0}{1-\dfrac{V_0\,t}{H_0}}.$$

Pour $\rho_{\mathrm{air},0}=1.176829268\,\mathrm{kg/m}^3$, on obtient à $t=0.3\,\mathrm{s}$: $\rho_{\mathrm{th}}=1.9613821138\,\mathrm{kg/m}^3$ ($p_{\mathrm{th}}=168\,875\,\mathrm{Pa}$) ; et à $t=0.5\,\mathrm{s}$: $\rho_{\mathrm{th}}=3.530487804\,\mathrm{kg/m}^3$ ($p_{\mathrm{th}}=303\,975\,\mathrm{Pa}$).

10.3 3. Paramètres Physiques, Numériques et Discrétisation

Physiques, Numériques et Matrice des Jeux de Paramètres

Catégorie	Paramètre	Valeur / Spécification
Domaine et interface initiale	$[0,L_x]\times[0,L_y]$, h_0, H_0	$L_x=L_y=0.1\,\mathrm{m}$, eau sur $y\in[0,0.025]\,\mathrm{m}$, air sur $H_0=0.075\,\mathrm{m}$
Fluide 1 (Eau liquide)	$\rho_{1,0},\mu_1,\chi_{T,1}$	$\rho_{1,0}=1000.0\,\mathrm{kg/m}^3$, $\mu_1=10^{-3}\,\mathrm{Pa\cdot s}$, $\chi_{T,1}=0.444\times10^{-9}\,\mathrm{Pa}^{-1}$
Fluide 2 (Air compressible)	$\rho_{2,0},\mu_2,\chi_{T,2}$	$\rho_{2,0}=1.176829268\,\mathrm{kg/m}^3$, $\mu_2=1.85\times10^{-5}\,\mathrm{Pa\cdot s}$, $\chi_{T,2}=9.8692327\times10^{-6}\,\mathrm{Pa}^{-1}$
Conditions aux limites	Bas ($y=0$), Haut/Gauche/Droite	Injection `DIRICHLET` ($u=0,v=V_0=0.1\,\mathrm{m/s}$) en bas ; `FREE_SLIP` sur les 3 autres bords
Séquence de grilles et Δt	$N\in\{32,64,128,256,512\}$	$\Delta t=10^{-3}\,\mathrm{s}$ ($t_{\mathrm{final}}=0.5\,\mathrm{s}$, soit 500 pas de temps ; contrôle intermédiaire à $t=0.3\,\mathrm{s}$)
Solveurs et schémas VOF	`solver` & `vof`	`MONOLITHIC` ($r_{\mathrm{al}}=0$ requis en compressible) avec `DIRECT`, `BICGSTAB2`, `GMRES` ; `WEYMOUTH_YUE`, `CIAM`, `VOF_TVD`

10.4 4. Fichier de Configuration JSON Pas à Pas

de Fichiers `.json` (C++17 et Fortran 2018)

10.4.1 Prototype `case/cpp/cas_10_piston_compressible_n128.json`

(Pour le moteur Fortran, remplacer `output/cpp/` par `output/fortran/` dans `output_dir`)

```
{
  "mode": "NAVIER_STOKES",
  "mesh": {
    "nx": 128,
    "ny": 128,
    "lx": 0.1,
    "ly": 0.1
  },
  "physics": {
    "enable_two_phase": true,
    "compressible": true,
    "enable_convection": true,
    "enable_surface_tension": false,
    "gx": 0.0,
    "gy": 0.0,
    "fluid1": { "rho": 1000.0, "mu": 1.0e-3, "chi_t": 4.44e-10 },
    "fluid2": { "rho": 1.176829268, "mu": 1.85e-5, "chi_t": 9.8692327e-6 }
  },
  "initial_condition": {
    "type": "DAM_BREAK",
    "box_x0": 0.0,
    "box_y0": 0.0,
    "box_x1": 0.1,
    "box_y1": 0.025,
    "p_init": 101325.0,
    "u0": 0.0,
    "v0": 0.0
  },
  "time": {
    "dt": 0.001,
    "t_final": 0.5,
    "time_scheme": "EULER",
    "adaptive_dt": false
  },
  "boundary_conditions": {
    "left":   { "type": "FREE_SLIP", "u": 0.0, "v": 0.0 },
    "right":  { "type": "FREE_SLIP", "u": 0.0, "v": 0.0 },
    "bottom": { "type": "DIRICHLET", "u": 0.0, "v": 0.1 },
    "top":    { "type": "FREE_SLIP", "u": 0.0, "v": 0.0 }
  },
  "solver": {
    "type": "MONOLITHIC",
    "linear_solver": "BICGSTAB2",
    "tol": 1e-10,
    "max_iter": 1000,
    "r_al": 0.0,
    "use_precond": true,
    "precond_type": "BLOCK_TRIANGULAR"
  },
  "vof": {
    "advection_method": "WEYMOUTH_YUE",
    "reconstruction_method": "PLIC",
    "tvd_limiter": "SUPERBEE",
    "cfl_vof": 0.5
  },
  "output": {
    "output_dir": "VALIDATION_TOTALE/validation_cas_10_piston_compressible/output/cpp/n128",
    "case_name": "cas_10_piston_compressible_n128",
    "vtk_interval": 50,
    "log_interval": 10,
    "write_csv_history": true
  }
```

```
}
```

10.5 5. Procédure d'Exécution (Moteurs C++17 et Fortran 2018)

L'exécution s'effectue indifféremment à l'aide du binaire C++17 (`covof2d_cpp`) ou du binaire Fortran 2018 (`covof2d_fortran`), garantissant une stricte équivalence des résultats numériques.

```
# 1. Se placer dans le répertoire du cas
cd VALIDATION_TOTALE/Run_Valide-CASES/validation_cas_10_piston_compressible

# 2. Exécution avec le moteur C++17 (Séquentiel)
../../../COVOF_2D-C++/bin/covof2d_cpp case/cpp/cas_10_config.json

# 3. Exécution avec le moteur C++17 (Parallèle OpenMP)
export OMP_NUM_THREADS=4
../../../COVOF_2D-C++/bin/covof2d_cpp case/cpp/cas_10_config.json

# 4. Exécution avec le moteur Fortran 2018 (Séquentiel)
../../../COVOF_2D-FORTRAN/bin/covof2d_fortran case/fortran/cas_10_config.json

# 5. Exécution avec le moteur Fortran 2018 (Parallèle OpenMP)
export OMP_NUM_THREADS=4
../../../COVOF_2D-FORTRAN/bin/covof2d_fortran case/fortran/cas_10_config.json
```

10.6 6. Analyse Quantitative, Script Python et Visualisation ParaView

10.6.1 6.1 Critères de Vérification et Tolérances

à Mener (Précision, Intégrité et Performance)

1. **Précision sur la loi de compression analytique $\rho_{\text{air}}(t)$ et $p(t)$:** Extraire la masse volumique moyenne du gaz $\overline{\rho}_{\text{air}}(t)$ et la pression moyenne $\overline{p}_{\text{air}}(t)$ dans la zone $C < 0.01$ aux instants $t \in \{0.1, 0.2, 0.3, 0.4, 0.5\}$ s. Vérifier qu'à $t = 0.3$ s, $\overline{\rho}_{\text{air}}$ converge vers $\rho_{\text{th}} = 1.9613821138\,\text{kg/m}^3$ avec une erreur relative décroissant de $0.50\,\%$ (32^2) à $0.031\,\%$ (512^2), soit un ordre de convergence spatial $p = 1.00$. Vérifier qu'à $t = 0.5$ s, $\overline{\rho}_{\text{air}} \approx 3.5305\,\text{kg/m}^3$ et $\overline{p}_{\text{air}} \approx 303\,967$ Pa (écart $< 0.01\,\%$).
2. **Intégrité du bloc diagonal compressible $C_p = -\frac{\chi_T}{\Delta t} I$ et des solveurs :** En mode `compressible: true`, le bloc $(2,2)$ du système selle vaut $C_{p,i,j} = -(C_{i,j}\chi_{T,1} + (1 - C_{i,j})\chi_{T,2})/\Delta t$, ce qui lève la singularité de jauge de pression sans ancrage artificiel. Vérifier que `MONOLITHIC` ($r_{\text{al}} = 0$) résolu par `DIRECT`, `BICGSTAB2`, `BICGSTAB` et `GMRES` avec `BLOCK_TRIANGULAR`, `ILU0` et `SSOR` restitue exactement la même courbe $p(t)$ à 10^{-8} Pa près.
3. **Comparaison des schémas VOF avec dilatation $C\nabla\cdot\mathbf{u}$ et parité dual-code :** Comparer `WEYMOUTH_YUE+PLIC`, `CIAM+PLIC` et `VOF_TVD` sur la planéité de l'interface $y_{\text{int}}(t) = 0.025 + 0.1t$ et la conservation de la masse totale d'air $M_{\text{air}}(t) = M_{\text{air}}(0)$. Vérifier la parité entre `covof2d_cpp` et `covof2d_fortran` ($\Delta_{\text{dual}}(p) < 10^{-8}$ Pa).

10.6.2 6.2 Résultats Attendus et Visualisation ParaView

Chapitre 11

Tutoriel 11 : Compression Isotrope d'une Bulle d'Air en Cavité Carrée

Ce tutoriel évalue la capacité de l'algorithme C-AAL à traiter la compression axisymétrique ou uniforme d'une bulle de gaz immergée. L'évolution temporelle du rayon de la bulle et de la densité de la phase compressible est confrontée à la solution exacte déduite du bilan de masse global.

11.1 1. Objectifs Pédagogiques et Modules Logiciels Activés

11.1.1 1.1 Compétences Acquises

- Comprendre les fondements physiques et les équations aux dérivées partielles gouvernant l'écoulement.
- Configurer les paramètres numériques et conditions aux limites sur grille cartésienne décalée MAC.
- Exécuter le cas test avec les exécutables C++17 et Fortran 2018 (séquentiel et OpenMP).
- Quantifier avec rigueur la précision par rapport à la solution analytique exacte (L_2, L_∞).
- Produire et exploiter les visualisations scientifiques volumiques sous ParaView.

11.1.2 1.2 Modules Logiciels Sollicités

- **Moteur C++17 :** `Mesh2D`, `FlowFields`, `CAALSolver`, `VOFCompressible`, `BiCGSTAB2Solver`.
- **Moteur Fortran 2018 :** `module_mesh`, `module_flow_fields`, `module_caal`, `module_vof_compressible`, `module_linear_solvers`.

11.2 2. Formulation Physique et Solution Analytique Exacte

Équations Physiques et Configuration

Ce treizième cas valide la compression multidimensionnelle centripète d'une bulle d'air circulaire immergée dans de l'eau au moyen du modèle 1-fluide faiblement compressible (`compressible: true`).

Dans une cavité carrée $\Omega = [0, 0.1] \times [0, 0.1]\,\mathrm{m}^2$ sans gravité ($\mathbf{g} = \mathbf{0}$), une bulle d'air de rayon initial $R_0 = 0.03\,\mathrm{m}$ ($C = 0$, $\rho_{2,0} = 1.176829268\,\mathrm{kg/m}^3$, $\mu_2 = 1.85 \times 10^{-5}\,\mathrm{Pa \cdot s}$, $\chi_{T,2} = 9.8692327 \times 10^{-6}\,\mathrm{Pa}^{-1}$) est centrée en $(0.05, 0.05)\,\mathrm{m}$ dans de l'eau ($C = 1$, $\rho_{1,0} = 1000\,\mathrm{kg/m}^3$, $\mu_1 = 10^{-3}\,\mathrm{Pa \cdot s}$, $\chi_{T,1} = 0.444 \times 10^{-9}\,\mathrm{Pa}^{-1}$). Sur chacune des 4 parois, de l'eau est injectée normalement vers l'intérieur avec une vitesse $V_0 = 0.0025\,\mathrm{m/s}$, soit un débit entrant total $q = 4LV_0 = 1.0 \times 10^{-3}\,\mathrm{m}^2/\mathrm{s}$.

11.2.0.1 Loi analytique d'évolution du rayon, de la masse volumique et de la pression

L'aire de la bulle décroît linéairement selon $\pi R^2(t) = \pi R_0^2 - q\,t$ (temps d'effondrement complet $t_{\text{collapse}} = \pi R_0^2/q = 2.827433388\,\text{s}$), d'où :

$$R(t) = R_0\sqrt{1 - \frac{q\,t}{\pi R_0^2}}, \qquad \rho_{\text{air}}(t) = \frac{\rho_{\text{air},0}}{1 - \dfrac{q\,t}{\pi R_0^2}}, \qquad p(t) = \frac{p_0}{1 - \dfrac{q\,t}{\pi R_0^2}}.$$

À $t = 1.5\,\text{s}$ ($\Delta t = 0.00125\,\text{s}$), on obtient $\rho_{\text{th}}(1.5\,\text{s}) = 2.50664658\,\text{kg/m}^3$, $R_{\text{th}}(1.5\,\text{s}) = 0.0205543\,\text{m}$ et $p_{\text{th}}(1.5\,\text{s}) = 215\,821.8\,\text{Pa}$.

11.3 3. Paramètres Physiques, Numériques et Discrétisation

Physiques, Numériques et Matrice des Jeux de Paramètres

Catégorie	Paramètre	Valeur / Spécification
Domaine et bulle initiale	$[0, L]^2$, (x_c, y_c), R_0	$L = 0.1\,\text{m}$, centre $(0.05, 0.05)\,\text{m}$, $R_0 = 0.03\,\text{m}$, $p_0 = 101\,325\,\text{Pa}$
Fluides 1 (Eau) et 2 (Air)	$\rho_{i,0}, \mu_i, \chi_{T,i}$	Eau : $\rho_{1,0} = 1000$, $\chi_{T,1} = 4.44 \times 10^{-10}$; Air : $\rho_{2,0} = 1.176829268$, $\chi_{T,2} = 9.8692327 \times 10^{-6}\,\text{Pa}^{-1}$
Conditions d'injection (4 bords)	Gauche, Droite, Bas, Haut	`DIRICHLET` entrant : $u_{\text{left}} = +V_0$, $u_{\text{right}} = -V_0$, $v_{\text{bottom}} = +V_0$, $v_{\text{top}} = -V_0$ ($V_0 = 0.0025\,\text{m/s}$)
Séquence de grilles et Δt	$N \in \{32, 64, 128, 256, 512\}$	$\Delta t = 0.00125\,\text{s}$, $t_{\text{final}} = 1.5\,\text{s}$ (1200 pas) et test prolongé jusqu'à $t = 2.5\,\text{s}$
Solveurs et schémas VOF	`solver` & `vof`	`MONOLITHIC` avec `BICGSTAB2` + `BLOCK_TRIANGULAR` / `DIRECT` ; `WEYMOUTH_YUE+PLIC`, `CIAM+PLIC`, `VOF_TVD`

11.4 4. Fichier de Configuration JSON Pas à Pas

de Fichiers `.json` (C++17 et Fortran 2018)

11.4.1 Prototype `case/cpp/cas_11_compression_bulle_n128.json`

(Pour le moteur Fortran, remplacer `output/cpp/` par `output/fortran/` dans `output_dir`)

```
{
  "mode": "NAVIER_STOKES",
  "mesh": {
    "nx": 128,
    "ny": 128,
    "lx": 0.1,
    "ly": 0.1
  },
  "physics": {
    "enable_two_phase": true,
    "compressible": true,
    "enable_convection": true,
    "enable_surface_tension": false,
```

```
    "gx": 0.0,
    "gy": 0.0,
    "fluid1": { "rho": 1000.0, "mu": 1.0e-3, "chi_t": 4.44e-10 },
    "fluid2": { "rho": 1.176829268, "mu": 1.85e-5, "chi_t": 9.8692327e-6 }
  },
  "initial_condition": {
    "type": "BUBBLE",
    "center_x": 0.05,
    "center_y": 0.05,
    "radius": 0.03,
    "p_init": 101325.0
  },
  "time": {
    "dt": 0.00125,
    "t_final": 1.5,
    "time_scheme": "EULER",
    "adaptive_dt": false
  },
  "boundary_conditions": {
    "left":   { "type": "DIRICHLET", "u":  0.0025, "v":  0.0 },
    "right":  { "type": "DIRICHLET", "u": -0.0025, "v":  0.0 },
    "bottom": { "type": "DIRICHLET", "u":  0.0,    "v":  0.0025 },
    "top":    { "type": "DIRICHLET", "u":  0.0,    "v": -0.0025 }
  },
  "solver": {
    "type": "MONOLITHIC",
    "linear_solver": "BICGSTAB2",
    "tol": 1e-10,
    "max_iter": 1000,
    "r_al": 0.0,
    "use_precond": true,
    "precond_type": "BLOCK_TRIANGULAR"
  },
  "vof": {
    "advection_method": "WEYMOUTH_YUE",
    "reconstruction_method": "PLIC",
    "tvd_limiter": "SUPERBEE",
    "cfl_vof": 0.5
  },
  "output": {
    "output_dir": "VALIDATION_TOTALE/validation_cas_11_compression_bulle/output/cpp/n128",
    "case_name": "cas_11_compression_bulle_n128",
    "vtk_interval": 100,
    "log_interval": 20,
    "write_csv_history": true
  }
}
```

11.5 5. Procédure d'Exécution (Moteurs C++17 et Fortran 2018)

L'exécution s'effectue indifféremment à l'aide du binaire C++17 (`covof2d_cpp`) ou du binaire Fortran 2018 (`covof2d_fortran`), garantissant une stricte équivalence des résultats numériques.

```
# 1. Se placer dans le répertoire du cas
cd VALIDATION_TOTALE/Run_Valide-CASES/validation_cas_11_compression_bulle

# 2. Exécution avec le moteur C++17 (Séquentiel)
../../../COVOF_2D-C++/bin/covof2d_cpp case/cpp/cas_11_config.json

# 3. Exécution avec le moteur C++17 (Parallèle OpenMP)
export OMP_NUM_THREADS=4
```

```
../../../COVOF_2D-C++/bin/covof2d_cpp case/cpp/cas_11_config.json

# 4. Exécution avec le moteur Fortran 2018 (Séquentiel)
../../../COVOF_2D-FORTRAN/bin/covof2d_fortran case/fortran/cas_11_config.json

# 5. Exécution avec le moteur Fortran 2018 (Parallèle OpenMP)
export OMP_NUM_THREADS=4
../../../COVOF_2D-FORTRAN/bin/covof2d_fortran case/fortran/cas_11_config.json
```

11.6 6. Analyse Quantitative, Script Python et Visualisation ParaView

11.6.1 6.1 Critères de Vérification et Tolérances

à Mener (Précision, Intégrité et Performance)

1. **Convergence vers** $\rho_{\text{th}}(1.5\,\text{s}) = 2.50664658\,\text{kg/m}^3$ **:** Évaluer $\overline{\rho}_{\text{air}}(1.5\,\text{s})$ sur $32^2, 64^2, 128^2, 256^2, 512^2$. Vérifier que l'erreur relative décroît de $1.96\,\%$ (32^2) et $0.45\,\%$ (64^2) à $0.015\,\%$ (512^2), avec un ordre global $p \approx 1.76 - 2.00$.
2. **Circularité de la bulle comprimée et bilan acoustique global :** Vérifier que $\int_\Omega \chi_T(C) \frac{p^{n+1}-p^n}{\Delta t}\,\mathrm{d}\Omega = q$ absorbe exactement le débit injecté sur les 4 parois, et comparer la circularité du contour $C = 0.5$ entre `WEYMOUTH_YUE+PLIC`, `CIAM+PLIC` et `VOF_TVD`.
3. **Performance des préconditionneurs et parité dual-code :** Comparer `BLOCK_TRIANGULAR`, `ILU0` et `SSOR` avec `BICGSTAB2` et `GMRES`, et vérifier la parité C++17 / Fortran 2018.

11.6.2 6.2 Résultats Attendus et Visualisation ParaView

Chapitre 12

Tutoriel 12 : Rupture de Barrage 2D sur Fond Sec (Solution de Saint-Venant)

Ce tutoriel aborde l'effondrement gravitationnel instantané d'une colonne d'eau sur fond sec. La cinématique d'avancée du front sec et le profil parabolique de la surface libre sont confrontés à la solution exacte issue de la théorie des caractéristiques de Saint-Venant (solution de Ritter).

12.1 1. Objectifs Pédagogiques et Modules Logiciels Activés

12.1.1 1.1 Compétences Acquises

- Comprendre les fondements physiques et les équations aux dérivées partielles gouvernant l'écoulement.
- Configurer les paramètres numériques et conditions aux limites sur grille cartésienne décalée MAC.
- Exécuter le cas test avec les exécutables C++17 et Fortran 2018 (séquentiel et OpenMP).
- Quantifier avec rigueur la précision par rapport à la solution analytique exacte (L_2, L_∞).
- Produire et exploiter les visualisations scientifiques volumiques sous ParaView.

12.1.2 1.2 Modules Logiciels Sollicités

- **Moteur C++17 :** `Mesh2D`, `FlowFields`, `WeymouthYueVOF`, `NavierStokesSolver`, `UzawaSolver`.
- **Moteur Fortran 2018 :** `module_mesh`, `module_flow_fields`, `module_vof`, `module_navier_stokes`, `module_uzawa`.

12.2 2. Formulation Physique et Solution Analytique Exacte

Équations Physiques et Configuration

Ce vingtième cas valide le couplage Navier-Stokes diphasique à surface libre ($\rho_1/\rho_2 \approx 850$) sous gravité sur le problème de la rupture de barrage 2D sur fond sec (*Dam Break*).

Dans un canal $\Omega = [0, 1.2] \times [0, 0.2]\,\mathrm{m}^2$ à parois glissantes (`FREE_SLIP`), une colonne d'eau occupe initialement $[0, 0.6] \times [0, 0.1]\,\mathrm{m}^2$ ($x_0 = 0.6\,\mathrm{m}, h_g = 0.1\,\mathrm{m}$) avec $\rho_1 = 1000\,\mathrm{kg/m}^3, \mu_1 = 10^{-3}\,\mathrm{Pa}\cdot\mathrm{s}$,

$\rho_2 = 1.1768\,\mathrm{kg/m^3}$, $\mu_2 = 10^{-5}\,\mathrm{Pa\cdot s}$, $\sigma = 0.075\,\mathrm{N/m}$ et $g_y = -9.81\,\mathrm{m/s^2}$.

12.2.0.1 Solution analytique des caractéristiques de Saint-Venant

Avec la célérité d'onde $c_0 = \sqrt{g h_g} = 0.990454\,\mathrm{m/s}$, la hauteur d'eau analytique en eau peu profonde vaut :

$$h_{\mathrm{SV}}(x,t) = \begin{cases} h_g & \text{si } x \le x_0 - c_0 t, \\ \dfrac{1}{9g}\left(2c_0 - \dfrac{x - x_0}{t}\right)^2 & \text{si } x_0 - c_0 t < x < x_0 + 2c_0 t, \\ 0 & \text{si } x \ge x_0 + 2c_0 t. \end{cases}$$

Les profils de surface libre $h_{\mathrm{num}}(x,t) = \int_0^{L_y} C(x,y,t)\,\mathrm{d}y$ sont comparés à $h_{\mathrm{SV}}(x,t)$ aux instants $t = 0.24\,\mathrm{s}$ et $t = 0.40\,\mathrm{s}$.

12.3 3. Paramètres Physiques, Numériques et Discrétisation

Physiques, Numériques et Matrice des Jeux de Paramètres

Catégorie	Paramètre	Valeur / Spécification
Domaine et retenue initiale	$[0,1.2]\times[0,0.2]\,\mathrm{m}^2$, x_0, h_g	$x_0 = 0.6\,\mathrm{m}$, $h_g = 0.1\,\mathrm{m}$ (`DAM_BREAK`), $g_y = -9.81\,\mathrm{m/s^2}$
Fluides 1 (Eau) et 2 (Air)	$\rho_1, \mu_1, \rho_2, \mu_2, \sigma$	$\rho_1 = 1000\,\mathrm{kg/m^3}$, $\mu_1 = 10^{-3}\,\mathrm{Pa\cdot s}$; $\rho_2 = 1.1768\,\mathrm{kg/m^3}$, $\mu_2 = 10^{-5}\,\mathrm{Pa\cdot s}$; $\sigma = 0.075\,\mathrm{N/m}$
Séquence de grilles et Δt	120×20, 240×40, 480×80	$\Delta t \in \{10^{-3}, 5\times10^{-4}, 2.5\times10^{-4}\}\,\mathrm{s}$, $t_{\mathrm{final}} = 0.50\,\mathrm{s}$ (instants clés 0.24 s et 0.40 s)
Schémas VOF et solveurs	`vof` & `solver`	`WEYMOUTH_YUE+PLIC`, `CIAM+PLIC`, `VOF_TVD` (`SUPERBEE`) ; `ADAPTATIVE_AUGMENTED_LAGRANGIAN`, `AUGMENTED_LAGRANGIAN`, `MONOLITHIC`

12.4 4. Fichier de Configuration JSON Pas à Pas

de Fichiers `.json` (C++17 et Fortran 2018)

12.4.1 Prototype `case/cpp/cas_12_rupture_barrage_240x40.json`

(Pour le moteur Fortran, remplacer `output/cpp/` par `output/fortran/` dans `output_dir`)

```
{
  "mode": "NAVIER_STOKES",
  "mesh": {
    "nx": 240,
    "ny": 40,
    "lx": 1.2,
    "ly": 0.2
  },
  "physics": {
    "enable_two_phase": true,
    "enable_convection": true,
    "convective_scheme": "TVD_SUPERBEE",
```

```
    "convective_linearization": "ADAMS_BASHFORTH",
    "enable_surface_tension": true,
    "enable_diffusion_smoothing": true,
    "diffusion_smoothing_steps": 2,
    "sigma": 0.075,
    "gx": 0.0,
    "gy": -9.81,
    "fluid1": { "rho": 1000.0, "mu": 1.0e-3 },
    "fluid2": { "rho": 1.1768, "mu": 1.0e-5 }
  },
  "initial_condition": {
    "type": "DAM_BREAK",
    "box_x0": 0.0,
    "box_y0": 0.0,
    "box_x1": 0.6,
    "box_y1": 0.1
  },
  "time": {
    "dt": 0.0005,
    "t_final": 0.50,
    "time_scheme": "GEAR",
    "adaptive_dt": false
  },
  "boundary_conditions": {
    "left":   { "type": "FREE_SLIP", "u": 0.0, "v": 0.0 },
    "right":  { "type": "FREE_SLIP", "u": 0.0, "v": 0.0 },
    "bottom": { "type": "FREE_SLIP", "u": 0.0, "v": 0.0 },
    "top":    { "type": "FREE_SLIP", "u": 0.0, "v": 0.0 }
  },
  "solver": {
    "type": "ADAPTATIVE_AUGMENTED_LAGRANGIAN",
    "adaptative_al": true,
    "k_aal": 100.0,
    "uzawa_tol": 1e-6,
    "uzawa_iter": 5,
    "linear_solver": "BICGSTAB2",
    "tol": 1e-08,
    "max_iter": 800,
    "use_precond": true,
    "precond_type": "BLOCK_TRIANGULAR"
  },
  "vof": {
    "advection_method": "WEYMOUTH_YUE",
    "reconstruction_method": "PLIC",
    "tvd_limiter": "SUPERBEE",
    "cfl_vof": 0.3
  },
  "output": {
    "output_dir": "VALIDATION_TOTALE/validation_cas_12_rupture_barrage/output/cpp/grid_240x40",
    "case_name": "cas_12_rupture_barrage_240x40",
    "vtk_interval": 40,
    "log_interval": 10,
    "write_csv_history": true
  }
}
```

12.5 5. Procédure d'Exécution (Moteurs C++17 et Fortran 2018)

L'exécution s'effectue indifféremment à l'aide du binaire C++17 (`covof2d_cpp`) ou du binaire Fortran 2018 (`covof2d_fortran`), garantissant une stricte équivalence des résultats numériques.

```
# 1. Se placer dans le répertoire du cas
cd VALIDATION_TOTALE/Run_Valide-CASES/validation_cas_12_rupture_barrage

# 2. Exécution avec le moteur C++17 (Séquentiel)
../../../COVOF_2D-C++/bin/covof2d_cpp case/cpp/cas_12_config.json

# 3. Exécution avec le moteur C++17 (Parallèle OpenMP)
export OMP_NUM_THREADS=4
../../../COVOF_2D-C++/bin/covof2d_cpp case/cpp/cas_12_config.json

# 4. Exécution avec le moteur Fortran 2018 (Séquentiel)
../../../COVOF_2D-FORTRAN/bin/covof2d_fortran case/fortran/cas_12_config.json

# 5. Exécution avec le moteur Fortran 2018 (Parallèle OpenMP)
export OMP_NUM_THREADS=4
../../../COVOF_2D-FORTRAN/bin/covof2d_fortran case/fortran/cas_12_config.json
```

12.6 6. Analyse Quantitative, Script Python et Visualisation ParaView

12.6.1 6.1 Critères de Vérification et Tolérances

à Mener (Précision, Intégrité et Performance)

1. **Profils de surface libre $h(x,t)$ à $t = 0.24\,\mathrm{s}$ et $t = 0.40\,\mathrm{s}$ et front d'onde $x_{\mathrm{front}}(t)$:** Extraire $h(x,t) = \int_0^{L_y} C(x,y,t)\,\mathrm{d}y$ sur $120 \times 20, 240 \times 40, 480 \times 80$ et comparer à la solution analytique de Saint-Venant $h_{\mathrm{SV}}(x,t)$.
2. **Conservation du volume d'eau et comparaison des 3 schémas VOF :** Vérifier que $\Delta V/V_0 < 10^{-5}$ et comparer `WEYMOUTH_YUE+PLIC`, `CIAM+PLIC` et `VOF_TVD`.
3. **Performance de `AAL` vs `AL` et parité C++17 / Fortran 2018 :** Comparer les performances des solveurs et vérifier la parité C++/Fortran.

12.6.2 6.2 Résultats Attendus et Visualisation ParaView

Chapitre 13

Tutoriel 13 : Oscillations Capillaires Amorties d'une Bulle de Gaz (Théorie de Lamb)

Ce tutoriel met en œuvre une bulle initialement déformée en ellipse selon le mode n=2 sous l'action combinée de la tension superficielle et de la dissipation visqueuse. La fréquence propre d'oscillation et le taux d'amortissement sont confrontés à la théorie linéaire exacte de Lamb.

13.1 1. Objectifs Pédagogiques et Modules Logiciels Activés

13.1.1 1.1 Compétences Acquises

- Comprendre les fondements physiques et les équations aux dérivées partielles gouvernant l'écoulement.
- Configurer les paramètres numériques et conditions aux limites sur grille cartésienne décalée MAC.
- Exécuter le cas test avec les exécutables C++17 et Fortran 2018 (séquentiel et OpenMP).
- Quantifier avec rigueur la précision par rapport à la solution analytique exacte (L_2, L_∞).
- Produire et exploiter les visualisations scientifiques volumiques sous ParaView.

13.1.2 1.2 Modules Logiciels Sollicités

- **Moteur C++17 :** `Mesh2D`, `FlowFields`, `WeymouthYueVOF`, `CSFModel`, `NavierStokesSolver`.
- **Moteur Fortran 2018 :** `module_mesh`, `module_flow_fields`, `module_vof`, `module_csf`, `module_navier_stokes`.

13.2 2. Formulation Physique et Solution Analytique Exacte

Équations Physiques et Configuration

Ce douzième cas valide la dynamique instationnaire du couplage entre l'inertie diphasique à fort rapport de densité ($\rho_1/\rho_2 \approx 5950$), la dissipation visqueuse et la force de tension superficielle CSF lors des oscillations libres d'une bulle de gaz initialement elliptique.

Dans une cavité carrée $\Omega = [0, 0.1] \times [0, 0.1]\,\mathrm{m}^2$ sans gravité ($\mathbf{g} = \mathbf{0}$) avec parois à glissement libre (`FREE_SLIP`), une bulle d'air ($\rho_2 = 1.17683\,\mathrm{kg/m}^3$, $\mu_2 = 1.85 \times 10^{-5}\,\mathrm{Pa \cdot s}$) est centrée en $(0.05, 0.05)\,\mathrm{m}$

dans un métal liquide dense ($\rho_1 = 7000\,\mathrm{kg/m^3}$, $\mu_1 = 4.0 \times 10^{-3}\,\mathrm{Pa \cdot s}$) avec $\sigma = 1.8\,\mathrm{N/m}$. À $t = 0$, la bulle de rayon équivalent $R_0 = 0.02\,\mathrm{m}$ est déformée sur le mode quadrupolaire $n = 2$ ($a_0 = R_0(1+\varepsilon)$, $b_0 = R_0/(1+\varepsilon)$, avec $\varepsilon = 0.20$).

13.2.0.1 Relation de dispersion capillaire linéaire

La pulsation propre ω_n et la fréquence f_{th} du mode $n = 2$ en milieu infini valent :

$$\omega_n^2 = \frac{(n^3 - n)\,\sigma}{(\rho_1 + \rho_2)\,R_0^3} \implies \omega_2 = \sqrt{\frac{6\,\sigma}{(\rho_1 + \rho_2)\,R_0^3}}, \qquad f_{\mathrm{th}} = \frac{\omega_2}{2\pi} = 5.70632\,\mathrm{s^{-1}} \quad (T_{\mathrm{th}} = 0.17524\,\mathrm{s}).$$

Sur les grilles $32^2 \ldots 512^2$ en cavité confinée, la fréquence numérique croît de 4.368 Hz (32^2), 4.792 Hz (64^2), 4.952 Hz (128^2), 4.990 Hz (256^2) à 5.038 Hz (512^2).

13.3 3. Paramètres Physiques, Numériques et Discrétisation

Physiques, Numériques et Matrice des Jeux de Paramètres

Catégorie	Paramètre	Valeur / Spécification
Domaine et bulle initiale	$[0, L]^2$, (x_c, y_c), R_0, ε	$L = 0.1\,\mathrm{m}$, centre $(0.05, 0.05)\,\mathrm{m}$, $R_0 = 0.02\,\mathrm{m}$, $\varepsilon \in \{0.05, 0.20\}$
Fluide 1 (Métal liquide)	$\rho_1, \mu_1, \chi_{T,1}$	$\rho_1 = 7000.0\,\mathrm{kg/m^3}$, $\mu_1 = 4.0 \times 10^{-3}\,\mathrm{Pa \cdot s}$, $\chi_{T,1} = 10^{-9}\,\mathrm{Pa^{-1}}$
Fluide 2 (Gaz / Air)	$\rho_2, \mu_2, \chi_{T,2}$	$\rho_2 = 1.17683\,\mathrm{kg/m^3}$, $\mu_2 = 1.85 \times 10^{-5}\,\mathrm{Pa \cdot s}$, $\chi_{T,2} = 9.8692327 \times 10^{-6}\,\mathrm{Pa^{-1}}$
Tension superficielle et lissage	σ, `enable_diffusion_smoothing`	$\sigma = 1.8\,\mathrm{N/m}$, `enable_diffusion_smoothing: true`, `diffusion_smoothing_steps` $\in \{2, 3\}$
Séquence de grilles et Δt	$N \in \{32, 64, 128, 256, 512\}$	$\Delta t = 2.5 \times 10^{-4}\,\mathrm{s}$, $t_{\mathrm{final}} = 0.6\,\mathrm{s}$ (plus de 3 périodes complètes)
Solveurs et schémas VOF	`solver` & `vof`	`AUGMENTED_LAGRANGIAN`, `ADAPTATIVE_AUGMENTED_LAGRANGIAN`, `MONOLITHIC`; `WEYMOUTH_YUE+PLIC`, `CIAM+PLIC`

13.4 4. Fichier de Configuration JSON Pas à Pas

de Fichiers `.json` (C++17 et Fortran 2018)

13.4.1 Prototype `case/cpp/cas_13_oscillation_bulle_n128.json`

(Pour le moteur Fortran, remplacer ***`output/cpp/`*** *par* ***`output/fortran/`*** *dans* ***`output_dir`****)*

```
{
  "mode": "NAVIER_STOKES",
  "mesh": {
    "nx": 128,
    "ny": 128,
    "lx": 0.1,
```

```
    "ly": 0.1
  },
  "physics": {
    "enable_two_phase": true,
    "compressible": false,
    "enable_convection": true,
    "enable_surface_tension": true,
    "enable_diffusion_smoothing": true,
    "diffusion_smoothing_steps": 3,
    "sigma": 1.8,
    "gx": 0.0,
    "gy": 0.0,
    "fluid1": { "rho": 7000.0, "mu": 4.0e-3 },
    "fluid2": { "rho": 1.17683, "mu": 1.85e-5 }
  },
  "initial_condition": {
    "type": "BUBBLE",
    "center_x": 0.05,
    "center_y": 0.05,
    "radius": 0.02,
    "axis_a": 0.024,
    "axis_b": 0.0166666667
  },
  "time": {
    "dt": 0.00025,
    "t_final": 0.6,
    "time_scheme": "GEAR",
    "adaptive_dt": false
  },
  "boundary_conditions": {
    "left":   { "type": "FREE_SLIP", "u": 0.0, "v": 0.0 },
    "right":  { "type": "FREE_SLIP", "u": 0.0, "v": 0.0 },
    "bottom": { "type": "FREE_SLIP", "u": 0.0, "v": 0.0 },
    "top":    { "type": "FREE_SLIP", "u": 0.0, "v": 0.0 }
  },
  "solver": {
    "type": "ADAPTATIVE_AUGMENTED_LAGRANGIAN",
    "adaptative_al": true,
    "k_aal": 100.0,
    "uzawa_tol": 1e-6,
    "uzawa_iter": 6,
    "linear_solver": "BICGSTAB2",
    "tol": 1e-08,
    "max_iter": 600,
    "use_precond": true,
    "precond_type": "BLOCK_TRIANGULAR"
  },
  "vof": {
    "advection_method": "WEYMOUTH_YUE",
    "reconstruction_method": "PLIC",
    "tvd_limiter": "NONE",
    "cfl_vof": 0.3
  },
  "output": {
    "output_dir": "VALIDATION_TOTALE/validation_cas_13_oscillation_bulle_capillaire/output/cpp/n128",
    "case_name": "cas_13_oscillation_bulle_n128",
    "vtk_interval": 20,
    "log_interval": 5,
    "write_csv_history": true
  }
}
```

13.5 5. Procédure d'Exécution (Moteurs C++17 et Fortran 2018)

L'exécution s'effectue indifféremment à l'aide du binaire C++17 (`covof2d_cpp`) ou du binaire Fortran 2018 (`covof2d_fortran`), garantissant une stricte équivalence des résultats numériques.

```
# 1. Se placer dans le répertoire du cas
cd VALIDATION_TOTALE/Run_Valide-CASES/validation_cas_13_oscillation_bulle_capillaire

# 2. Exécution avec le moteur C++17 (Séquentiel)
../../../COVOF_2D-C++/bin/covof2d_cpp case/cpp/cas_13_config.json

# 3. Exécution avec le moteur C++17 (Parallèle OpenMP)
export OMP_NUM_THREADS=4
../../../COVOF_2D-C++/bin/covof2d_cpp case/cpp/cas_13_config.json

# 4. Exécution avec le moteur Fortran 2018 (Séquentiel)
../../../COVOF_2D-FORTRAN/bin/covof2d_fortran case/fortran/cas_13_config.json

# 5. Exécution avec le moteur Fortran 2018 (Parallèle OpenMP)
export OMP_NUM_THREADS=4
../../../COVOF_2D-FORTRAN/bin/covof2d_fortran case/fortran/cas_13_config.json
```

13.6 6. Analyse Quantitative, Script Python et Visualisation ParaView

13.6.1 6.1 Critères de Vérification et Tolérances

à Mener (Précision, Intégrité et Performance)

1. **Fréquence d'oscillation quadrupolaire f_{num} et convergence en maillage :** Extraire les demi-axes horizontal $a(t)$ et vertical $b(t)$ du contour $C = 0.5$ ainsi que l'énergie cinétique $E_k(t)$ dans `*_history.csv`. Vérifier la convergence de f_{num} sur $32^2 \ldots 512^2$ ($4.368 \to 4.792 \to 4.952 \to 4.990 \to 5.038$ Hz pour $\varepsilon = 0.20$).
2. **Robustesse du couplage vitesse-pression à haut rapport de densité ($\rho_1/\rho_2 \approx 5950$) :** Comparer `ADAPTATIVE_AUGMENTED_LAGRANGIAN` (`k_aal=100`), `AUGMENTED_LAGRANGIAN` ($r_{\text{al}} = 1000$) et `MONOLITHIC` avec `GEAR` vs `EULER`. Vérifier le gain d'itérations procuré par `AAL` et `BLOCK_-TRIANGULAR`.
3. **Conservation du volume de la bulle et parité C++17 / Fortran 2018 :** Vérifier que l'erreur relative sur l'aire de la bulle reste $< 10^{-4}$ avec `WEYMOUTH_YUE+PLIC` et que les courbes $a(t), b(t), E_k(t)$ C++ et Fortran se superposent.

13.6.2 6.2 Résultats Attendus et Visualisation ParaView

Chapitre 14

Tutoriel 14 : Instabilité Interfaciale Diphasique de Rayleigh-Taylor (Dispersion Linéaire)

Ce dernier tutoriel simule l'instabilité hydrodynamique se développant à l'interface entre un fluide lourd superposé à un fluide léger soumis à la gravité. Le taux de croissance exponentielle en régime linéaire est validé face à la relation de dispersion analytique exacte hydro-capillaire.

14.1 1. Objectifs Pédagogiques et Modules Logiciels Activés

14.1.1 1.1 Compétences Acquises

- Comprendre les fondements physiques et les équations aux dérivées partielles gouvernant l'écoulement.
- Configurer les paramètres numériques et conditions aux limites sur grille cartésienne décalée MAC.
- Exécuter le cas test avec les exécutables C++17 et Fortran 2018 (séquentiel et OpenMP).
- Quantifier avec rigueur la précision par rapport à la solution analytique exacte (L_2, L_∞).
- Produire et exploiter les visualisations scientifiques volumiques sous ParaView.

14.1.2 1.2 Modules Logiciels Sollicités

- **Moteur C++17 :** `Mesh2D`, `FlowFields`, `WeymouthYueVOF`, `CSFModel`, `NavierStokesSolver`, `UzawaSolver`.
- **Moteur Fortran 2018 :** `module_mesh`, `module_flow_fields`, `module_vof`, `module_csf`, `module_navier_stokes`, `module_uzawa`.

14.2 2. Formulation Physique et Solution Analytique Exacte

Équations Physiques et Configuration

Ce vingt-troisième cas valide la compétition entre l'instabilité gravitationnelle de Rayleigh-Taylor (fluide lourd au-dessus d'un fluide léger, $At = (\rho_1 - \rho_2)/(\rho_1 + \rho_2) = 0.5$) et la stabilisation capillaire CSF, du régime linéaire exponentiel jusqu'à l'enroulement non-linéaire en champignon.

Dans un domaine $\Omega = [0, 1.0] \times [0, 4.0]\,\mathrm{m}^2$ avec parois latérales `FREE_SLIP` (ou `PERIODIC`) et parois haute/basse `DIRICHLET` adhérentes, le fluide lourd ($\rho_1 = 3.0\,\mathrm{kg/m}^3, \mu_1 = 10^{-3}\,\mathrm{Pa\cdot s}$) surmonte initialement le fluide léger ($\rho_2 = 1.0\,\mathrm{kg/m}^3, \mu_2 = 10^{-3}\,\mathrm{Pa\cdot s}$) sous $g_y = -9.81\,\mathrm{m/s}^2$. L'interface initiale est perturbée selon (`type: "RAYLEIGH_TAYLOR"`, `center_y: 2.0`, `pert_amp: 0.05`, `pert_k: 1.0`) :

$$y_{\mathrm{int}}(x, 0) = 2.0 + A_0 \cos(kx), \qquad k = 2\pi\,\mathrm{m}^{-1}, \quad A_0 = 0.05\,\mathrm{m}.$$

14.2.0.1 Relation de dispersion hydro-capillaire linéaire

Le taux de croissance théorique $\omega_{\mathrm{th}}(k)$ et le seuil de coupure capillaire σ_c valent :

$$\omega_{\mathrm{th}}^2(k) = \frac{(\rho_1 - \rho_2)\, g\, k - \sigma\, k^3}{\rho_1 + \rho_2}, \qquad \sigma_c = \frac{(\rho_1 - \rho_2)\, g}{k^2} = 0.4970\,\mathrm{N/m}.$$

Pour $\sigma = 0$, $\omega_{\mathrm{th}} = 5.5515\,\mathrm{s}^{-1}$; pour $\sigma = 0.20\,\mathrm{N/m}$, $\omega_{\mathrm{th}} = 4.2915\,\mathrm{s}^{-1}$; et pour $\sigma = 0.60\,\mathrm{N/m} > \sigma_c$, l'interface est stabilisée et oscille sans croître.

14.3 3. Paramètres Physiques, Numériques et Discrétisation

Physiques, Numériques et Matrice des Jeux de Paramètres

Catégorie	Paramètre	Valeur / Spécification
Domaine et perturbation initiale	$[0, 1] \times [0, 4]\,\mathrm{m}^2$, y_c, A_0, k	$y_c = 2.0\,\mathrm{m}$, $A_0 = 0.05\,\mathrm{m}$, $m = 1$ ($k = 2\pi\,\mathrm{m}^{-1}$, `RAYLEIGH_TAYLOR`)
Fluides ($At = 0.5$) et gravité	$\rho_1, \rho_2, \mu_1, \mu_2, g_y$	$\rho_1 = 3.0\,\mathrm{kg/m}^3$, $\rho_2 = 1.0\,\mathrm{kg/m}^3$, $\mu_1 = \mu_2 = 10^{-3}\,\mathrm{Pa\cdot s}$, $g_y = -9.81\,\mathrm{m/s}^2$
Trois régimes capillaires σ	$\sigma \in \{0.0, 0.20, 0.60\}\,\mathrm{N/m}$	Instable pur ($\omega_{\mathrm{th}} = 5.55\,\mathrm{s}^{-1}$), ralenti ($\omega_{\mathrm{th}} = 4.29\,\mathrm{s}^{-1}$), stabilisé ($\sigma > \sigma_c = 0.497\,\mathrm{N/m}$)
Séquence de grilles et Δt	64×256, 128×512, 256×1024	$\Delta t = 10^{-3}\,\mathrm{s}$, $t_{\mathrm{final}} = 1.0\,\mathrm{s}$
Schémas VOF et solveurs	`vof` & `solver`	`WEYMOUTH_YUE+PLIC`, `CIAM+PLIC`, `VOF_TVD` ; `ADAPTATIVE_AUGMENTED_LAGRANGIAN`, `AUGMENTED_LAGRANGIAN`, `MONOLITHIC`

14.4 4. Fichier de Configuration JSON Pas à Pas

de Fichiers `.json` (C++17 et Fortran 2018)

14.4.1 Prototype `case/cpp/cas_14_rayleigh_taylor_sigma0_128x512.json`

(Pour le moteur Fortran, remplacer `output/cpp/` par `output/fortran/` dans `output_dir`)

```
{
  "mode": "NAVIER_STOKES",
  "mesh": {
    "nx": 128,
    "ny": 512,
    "lx": 1.0,
```

```
    "ly": 4.0
  },
  "physics": {
    "enable_two_phase": true,
    "enable_convection": true,
    "convective_scheme": "TVD_SUPERBEE",
    "convective_linearization": "ADAMS_BASHFORTH",
    "enable_surface_tension": false,
    "sigma": 0.0,
    "gx": 0.0,
    "gy": -9.81,
    "fluid1": { "rho": 3.0, "mu": 1.0e-3 },
    "fluid2": { "rho": 1.0, "mu": 1.0e-3 }
  },
  "initial_condition": {
    "type": "RAYLEIGH_TAYLOR",
    "center_y": 2.0,
    "pert_amp": 0.05,
    "pert_k": 1.0
  },
  "time": {
    "dt": 0.001,
    "t_final": 1.0,
    "time_scheme": "GEAR",
    "adaptive_dt": false
  },
  "boundary_conditions": {
    "left":   { "type": "FREE_SLIP", "u": 0.0, "v": 0.0 },
    "right":  { "type": "FREE_SLIP", "u": 0.0, "v": 0.0 },
    "bottom": { "type": "DIRICHLET", "u": 0.0, "v": 0.0 },
    "top":    { "type": "DIRICHLET", "u": 0.0, "v": 0.0 }
  },
  "solver": {
    "type": "ADAPTATIVE_AUGMENTED_LAGRANGIAN",
    "adaptative_al": true,
    "k_aal": 100.0,
    "uzawa_tol": 1e-6,
    "uzawa_iter": 4,
    "linear_solver": "BICGSTAB2",
    "tol": 1e-08,
    "max_iter": 800,
    "use_precond": true,
    "precond_type": "BLOCK_TRIANGULAR"
  },
  "vof": {
    "advection_method": "WEYMOUTH_YUE",
    "reconstruction_method": "PLIC",
    "tvd_limiter": "SUPERBEE",
    "cfl_vof": 0.3
  },
  "output": {
    "output_dir": "VALIDATION_TOTALE/validation_cas_14_instabilite_interface_-
    ↪  gravitaire/output/cpp/sigma0_128x512",
    "case_name": "cas_14_rayleigh_taylor_sigma0_128x512",
    "vtk_interval": 50,
    "log_interval": 10,
    "write_csv_history": true
  }
}
```

14.5 5. Procédure d'Exécution (Moteurs C++17 et Fortran 2018)

L'exécution s'effectue indifféremment à l'aide du binaire C++17 (`covof2d_cpp`) ou du binaire Fortran 2018 (`covof2d_fortran`), garantissant une stricte équivalence des résultats numériques.

```
# 1. Se placer dans le répertoire du cas
cd VALIDATION_TOTALE/Run_Valide-CASES/validation_cas_14_instabilite_interface_gravitaire

# 2. Exécution avec le moteur C++17 (Séquentiel)
../../../COVOF_2D-C++/bin/covof2d_cpp case/cpp/cas_14_config.json

# 3. Exécution avec le moteur C++17 (Parallèle OpenMP)
export OMP_NUM_THREADS=4
../../../COVOF_2D-C++/bin/covof2d_cpp case/cpp/cas_14_config.json

# 4. Exécution avec le moteur Fortran 2018 (Séquentiel)
../../../COVOF_2D-FORTRAN/bin/covof2d_fortran case/fortran/cas_14_config.json

# 5. Exécution avec le moteur Fortran 2018 (Parallèle OpenMP)
export OMP_NUM_THREADS=4
../../../COVOF_2D-FORTRAN/bin/covof2d_fortran case/fortran/cas_14_config.json
```

14.6 6. Analyse Quantitative, Script Python et Visualisation ParaView

14.6.1 6.1 Critères de Vérification et Tolérances

à Mener (Précision, Intégrité et Performance)

1. **Taux de croissance linéaire ω_{num} et seuil de stabilisation capillaire σ_c :** Ajuster $A(t) = A_0 \cosh(\omega t)$ sur $t \in [0.05, 0.30]\,\text{s}$ pour $\sigma \in \{0.0, 0.20, 0.60\}\,\text{N/m}$ et vérifier l'accord avec $\omega_{\text{th}}(0) = 5.5515\,\text{s}^{-1}$, $\omega_{\text{th}}(0.2) = 4.2915\,\text{s}^{-1}$ et la stabilisation capillaire pour $\sigma = 0.60 > \sigma_c$.
2. **Enroulement non-linéaire en champignon ($t \in [0.5, 1.0]\,\text{s}$), symétrie et conservation de masse :** Vérifier la symétrie $C(L_x - x, y, t) = C(x, y, t)$, $\Delta V/V_0 < 10^{-5}$, et comparer `WEYMOUTH_YUE+PLIC`, `CIAM+PLIC` et `VOF_TVD`.
3. **Performance des solveurs (`AAL`, `AL`, `MONOLITHIC`) et parité C++17 / Fortran 2018 :** Comparer les temps CPU et vérifier la superposition des interfaces C++ et Fortran.

14.6.2 6.2 Résultats Attendus et Visualisation ParaView